

Windows Phone Silverlight code that stores app settings as key-value pairs in isolated storage is easily ported. Here is a before-and-after example, first the Windows Phone Silverlight version:

C#

```
var propertySet = IsolatedStorageSettings.ApplicationSettings;
const string key = "favoriteAuthor";
propertySet[key] = "Charles Dickens";
propertySet.Save();
string myFavoriteAuthor = propertySet.Contains(key) ?
(string)propertySet[key] : "<none>";
```

And the UWP equivalent:

C#

```
var propertySet =
Windows.Storage.ApplicationData.Current.LocalSettings.Values;
const string key = "favoriteAuthor";
propertySet[key] = "Charles Dickens";
string myFavoriteAuthor = propertySet.ContainsKey(key) ?
(string)propertySet[key] : "<none>";
```

Although a subset of the **Windows.Storage** namespace is available to them, many Windows Phone Silverlight apps perform file i/o with the **IsolatedStorageFile** class because it has been supported for longer. Assuming that **IsolatedStorageFile** is being used, here's a before-and-after example of writing and reading a file, first the Windows Phone Silverlight version:

C#

```
const string filename = "FavoriteAuthor.txt";
using (var store = IsolatedStorageFile.GetUserStoreForApplication())
{
    using (var streamWriter = new
StreamWriter(store.CreateFile(filename)))
    {
        streamWriter.Write("Charles Dickens");
    }
    using (var StreamReader = new StreamReader(store.OpenFile(filename,
 FileMode.Open, FileAccess.Read)))
    {
        string myFavoriteAuthor = StreamReader.ReadToEnd();
    }
}
```

And the same functionality using the UWP:

C#

```
const string filename = "FavoriteAuthor.txt";
var store = Windows.Storage.ApplicationData.Current.LocalFolder;
Windows.Storage.StorageFile file = await store.CreateFileAsync(filename,
Windows.Storage.CreationCollisionOption.ReplaceExisting);
await Windows.Storage.FileIO.WriteTextAsync(file, "Charles Dickens");
file = await store.GetFileAsync(filename);
string myFavoriteAuthor = await
Windows.Storage.FileIO.ReadTextAsync(file);
```

A Windows Phone Silverlight app has read-only access to the optional SD card. A UWP app has read-write access to the SD card. For more info, see [Access the SD card](#).

For info about accessing photos, music, and video files in a UWP app, see [Files and folders in the Music, Pictures, and Videos libraries](#).

For more info, see [Files, folders, and libraries](#).

The next topic is [Porting for form factor and UX](#).

Related topics

- [Namespace and class mappings](#)

Porting Windows Phone Silverlight to UWP for form factor and UX

Article • 10/20/2022

The previous topic was [Porting business and data layers](#).

Windows apps share a common look-and-feel across PCs, mobile devices, and many other kinds of devices. The user interface, input, and interaction patterns are very similar, and a user moving between devices will welcome the familiar experience. Differences between devices such as physical size, default orientation, and effective pixel resolution factor into the way a Universal Windows Platform (UWP) app is rendered by Windows 10. The good news is that much of the heavy lifting is handled for you by the system using smart concepts such as effective pixels.

Different form factors and user experience

Different devices have multiple possible portrait and landscape resolutions, in a variety of aspect ratios. How will the visual aspects of its interface, text, and assets of your UWP app scale? How can you support touch as well as mouse and keyboard input? And with an app that supports touch on different-sized devices with different viewing distances, how can a control both be a right-sized touch target at different pixel densities *and* have its content readable at different distances? The following sections address the things you need to know.

What is the size of a screen, really?

The short answer is that it's subjective, because it depends not only on the objective size of the display but also on how far away from it you are. The subjectivity means that we have to put ourselves in the shoes of the user, and that's something that developers of good apps do in any case.

Objectively, a screen is measured in units of inches, and physical (raw) pixels. Knowing both of those metrics lets you know how many pixels fit into an inch. That's the pixel density, also known as DPI (dots per inch), also known as PPI (pixels per inch). And the reciprocal of the DPI is the physical size of the pixels as a fraction of an inch. Pixel density is also known as *resolution*, although that term is often used loosely to mean pixel count.

As viewing distance increases, all those objective metrics *seem* smaller, and they resolve into the screen's *effective size*, and its *effective resolution*. Your phone is usually held closest to your eye; your tablet then your PC monitor are next, and furthest away are [Surface Hub](#) devices and TVs. To compensate, devices tend to get objectively larger with viewing distance. When you set sizes on your UI elements, you are setting those sizes in units called effective pixels (epx). And Windows 10 will take into account DPI, and the typical viewing distance from a device, to calculate the best size of your UI elements in physical pixels to give the best viewing experience. See [View/effective pixels, viewing distance, and scale factors](#).

Even so, we recommend that you test your app with many different devices so that you can confirm each experience for yourself.

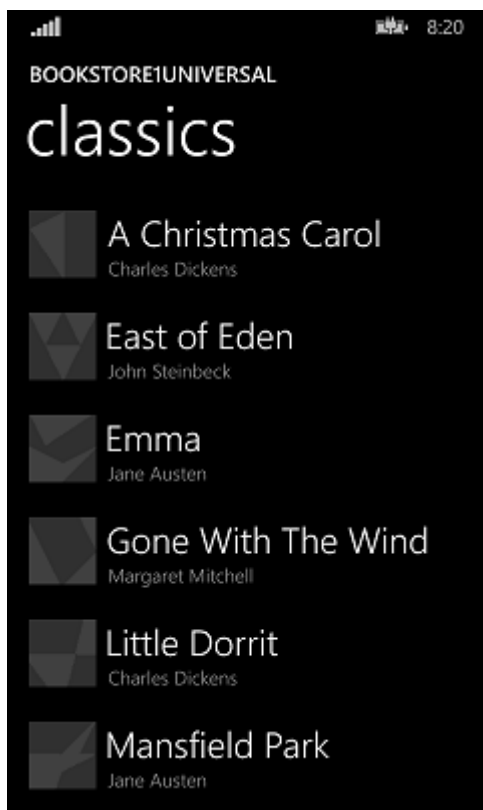
Touch resolution and viewing resolution

Affordances (UI widgets) need to be the right size to touch. So, a touch target needs to more-or-less retain its physical size across different devices that might have different pixel densities. Effective pixels help you out here, too: they're scaled on different devices—taking pixel density into account—in order to achieve a more-or-less constant physical size that's ideal for touch targets.

Text needs to be the right size to read (12 point text at a 20 inch viewing distance is a good rule of thumb), and an image needs to be the right size and effective resolution, for the viewing distance. On different devices, that same effective pixel scaling keeps UI elements right-sized and readable. Text and other vector-based graphics scale automatically, and very well. Raster (bitmap)-based graphics are also scaled automatically if the developer provides an asset in a single, large size. But, it's preferable for the developer to provide each asset in a range of sizes so that the appropriate one for a target device's scaling factor can be automatically loaded. For more info on that, see [View/effective pixels, viewing distance, and scale factors](#).

Layout, and adaptive Visual State Manager

We've described the factors involved in a meaningful understanding of screen size. Now, let's think about the layout of your app, and how to make use of extra screen real-estate when it's available. Consider this page from a very simple app that was designed to run on a narrow mobile device. What should this page look like on a larger screen?



The mobile version is constrained to portrait-only orientation because that's the best aspect ratio for the book list; and we'd do the same for a page of text, which is best kept to a single column on mobile devices. But, PC and tablet screens are large in either orientation, so that mobile device constraint seems like an unnecessary limitation on larger devices.

Optically zooming the app to look like the mobile version, just bigger, doesn't take advantage of the device and its additional space, and that doesn't serve the user well. We should be thinking of showing more content, rather than the same content bigger. Even on a phablet, we could show some more rows of content. We could use extra space to display different content, such as ads, or we could change the list box into a list view and have it wrap items into multiple columns, when it can, to use the space that way. See [Guidelines for list and grid view controls](#).

In addition to new controls such as list view and grid view, most of the established layout types from Windows Phone Silverlight have equivalents in the Universal Windows Platform (UWP). For example, [Canvas](#), [Grid](#), and [StackPanel](#). Porting much of the UI that uses these types should be straightforward, but always look for ways to leverage the dynamic layout capabilities of these layout panels to automatically resize and re-lay out on devices of different sizes.

Going beyond the dynamic layout built into the system controls and layout panels, we can use a new Windows 10 feature called [Adaptive Visual State Manager](#).

Input modalities

A Windows Phone Silverlight interface is touch-specific. And your ported app's interface should of course also support touch, but you can choose to support other input modalities in addition, such as mouse and keyboard. In the UWP, mouse, pen, and touch input are unified as *pointer input*. For more info, see [Handle pointer input](#), and [Keyboard interactions](#).

Maximizing markup and code re-use

Refer back to the [maximizing markup and code reuse](#) list for techniques on sharing your UI to target devices with a wide range of form factors.

More info and design guidelines

- [Design UWP apps](#)
- [Guidelines for fonts](#)
- [Plan for different form factors](#)

Related topics

- [Namespace and class mappings](#)

Windows Phone Silverlight to UWP case study: Bookstore1

Article • 10/20/2022

This topic presents a case study of porting a very simple Windows Phone Silverlight app to a Windows 10 Universal Windows Platform (UWP) app. With Windows 10, you can create a single app package that your customers can install onto a wide range of devices, and that's what we'll do in this case study. See [Guide to UWP apps](#).

The app we'll port consists of a **ListBox** bound to a view model. The view model has a list of books that shows title, author, and book cover. The book cover images have **Build Action** set to **Content** and **Copy to Output Directory** set to **Do not copy**.

The previous topics in this section describe the differences between the platforms, and they give details and guidance on the porting process for various aspects of an app from XAML markup, through binding to a view model, down to accessing data. A case study aims to complement that guidance by showing it in action in a real example. The case studies assume you've read the guidance, which they do not repeat.

Note When opening Bookstore1Universal_10 in Visual Studio, if you see the message "Visual Studio update required", then follow the steps for selecting a Target Platform Versioning in [TargetPlatformVersion](#).

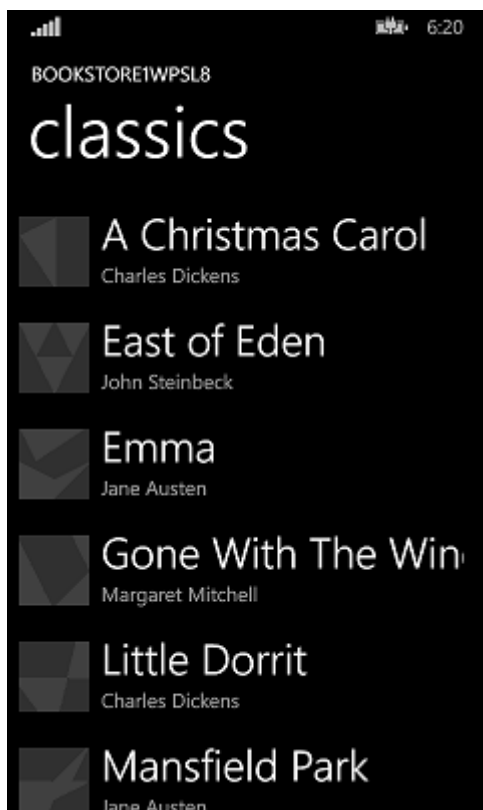
Downloads

[Download the Bookstore1WPSL8 Windows Phone Silverlight app](#) [↗](#).

[Download the Bookstore1Universal_10 Windows 10 app](#) [↗](#).

The Windows Phone Silverlight app

Here's what Bookstore1WPSL8—the app that we're going to port—looks like. It's just a vertically-scrolling list box of books beneath the heading of the app's name and page title.



Porting to a Windows 10 project

It's a very quick task to create a new project in Visual Studio, copy files over to it from Bookstore1WPSL8, and include the copied files in the new project. Start by creating a new Blank Application (Windows Universal) project. Name it Bookstore1Universal_10. These are the files to copy over from Bookstore1WPSL8 to Bookstore1Universal_10.

- Copy the folder containing the book cover image PNG files (the folder is \Assets\CoverImages). After copying the folder, in **Solution Explorer**, make sure **Show All Files** is toggled on. Right-click the folder that you copied and click **Include In Project**. That command is what we mean by "including" files or folders in a project. Each time you copy a file or folder, click **Refresh** in **Solution Explorer** and then include the file or folder in the project. There's no need to do this for files that you're replacing in the destination.
- Copy the folder containing the view model source file (the folder is \ViewModel).
- Copy MainPage.xaml and replace the file in the destination.

We can keep the App.xaml, and App.xaml.cs that Visual Studio generated for us in the Windows 10 project.

Edit the source code and markup files that you just copied and change any references to the Bookstore1WPSL8 namespace to Bookstore1Universal_10. A quick way to do that is to use the **Replace In Files** feature. In the imperative code in the view model source file, these porting changes are needed:

- Change `System.ComponentModel.DesignerProperties` to `DesignMode` and then use the **Resolve** command on it. Delete the `IsInDesignTool` property and use IntelliSense to add the correct property name: `DesignModeEnabled`.
- Use the **Resolve** command on `ImageSource`.
- Use the **Resolve** command on `BitmapImage`.
- Delete using `System.Windows.Media;` and using `System.Windows.Media.Imaging;`.
- Change the value returned by the `Bookstore1Universal_10.BookstoreViewModel.AppName` property from "BOOKSTORE1WPSL8" to "BOOKSTORE1UNIVERSAL".

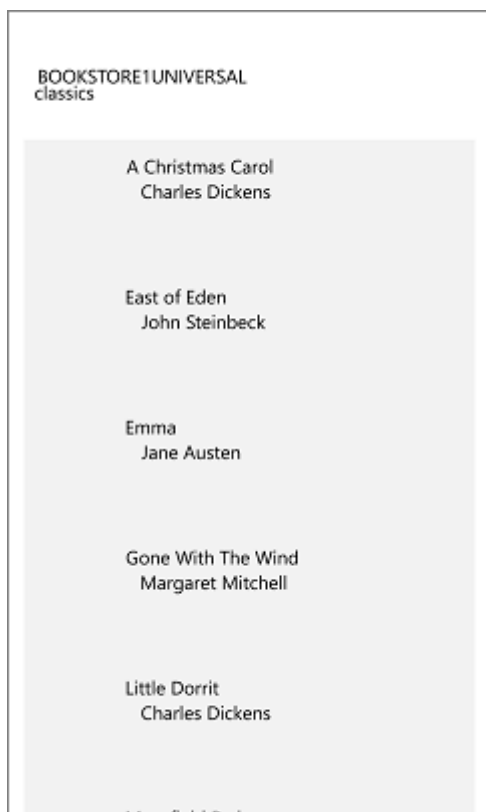
In `MainPage.xaml`, these porting changes are needed:

- Change `phone:PhoneApplicationPage` to `Page` (don't forget the occurrences in property element syntax).
- Delete the `phone` and `shell` namespace prefix declarations.
- Change "clr-namespace" to "using" in the remaining namespace prefix declaration.

We can choose to correct markup compilation errors very cheaply if we want to see results soonest, even if that means temporarily removing markup. But, let's keep a record of the debt we accrue by doing so. Here it is in this case.

1. In the root **Page** element in `MainPage.xaml`, delete `SupportedOrientations="Portrait"`.
2. In the root **Page** element in `MainPage.xaml`, delete `Orientation="Portrait"`.
3. In the root **Page** element in `MainPage.xaml`, delete `shell:SystemTray.IsVisible="True"`.
4. In the `BookTemplate` data template, delete the references to the `PhoneTextExtraLargeStyle` and `PhoneTextSubtleStyle` **TextBlock** styles.
5. In the `TitlePanel` **StackPanel**, delete the references to the `PhoneTextNormalStyle` and `PhoneTextTitle1Style` **TextBlock** styles.

Let's work on the UI for the mobile device family first, and we can consider other form factors after that. You can build and run the app now. Here's how it looks on the mobile emulator.



The view and the view model are working together correctly, and the **ListBox** is functioning. We mostly just need to fix the styling and get the images to show up.

Paying off the debt items, and some initial styling

By default, all orientations are supported. The Windows Phone Silverlight app explicitly constrains itself to portrait-only, though, so debt items #1 and #2 are paid off by going into the app package manifest in the new project and checking **Portrait** under **Supported orientations**.

For this app, item #3 is not a debt since the status bar (formerly called the system tray) is shown by default. For items #4 and #5, we need to find four Universal Windows Platform (UWP) **TextBlock** styles that correspond to the Windows Phone Silverlight styles that we were using. You can run the Windows Phone Silverlight app in the emulator and compare it side-by-side with the illustration in the [Text](#) section. From doing that, and from looking at the properties of the Windows Phone Silverlight system styles, we can make this table.

Windows Phone Silverlight style key	UWP style key
PhoneTextExtraLargeStyle	TitleTextBlockStyle
PhoneTextSubtleStyle	SubtitleTextBlockStyle

Windows Phone Silverlight style key	UWP style key
PhoneTextNormalStyle	CaptionTextBlockStyle
PhoneTextTitle1Style	HeaderTextBlockStyle

To set those styles, you can just type them into the markup editor or you can use the Visual Studio XAML Tools and set them without typing a thing. To do that, you right-click a **TextBlock** and click **Edit Style > Apply Resource**. To do that with the **TextBlocks** in the item template, right click the **ListBox** and click **Edit Additional Templates > Edit Generated Items (ItemTemplate)**.

There is an 80% opaque white background behind the items, because the default style of the **ListBox** control sets its background to the `ListBoxBackgroundThemeBrush` system resource. Set `Background="Transparent"` on the **ListBox** to clear that background. To left-align the **TextBlocks** in the item template, edit it again the same way as described above and set a **Margin** of `"9.6,0"` on both **TextBlocks**.

After that is done, because of [changes related to view pixels](#), we need to go through and multiply any fixed size dimension that we haven't yet changed (margins, width, height, etc) by 0.8. So, for example, the images should change from 70x70px to 56x56px.

But, let's get those images to render before we show the results of our styling.

Binding an Image to a view model

In Bookstore1WPSL8, we did this:

C#

```
// this.BookCoverImagePath contains a path of the form
"/Assets/CoverImages/one.png".
return new BitmapImage(new Uri(this.CoverImagePath, UriKind.Relative));
```

In Bookstore1Universal, we use the ms-appx [URI scheme](#). So that we can keep the rest of our code the same, we can use a different overload of the **System.Uri** constructor to put the ms-appx URI scheme in a base URI and append the rest of the path onto that. Like this:

C#

```
// this.BookCoverImagePath contains a path of the form
"/Assets/CoverImages/one.png".
```

```
return new BitmapImage(new Uri(new Uri("ms-appx://"),
this.CoverImagePath));
```

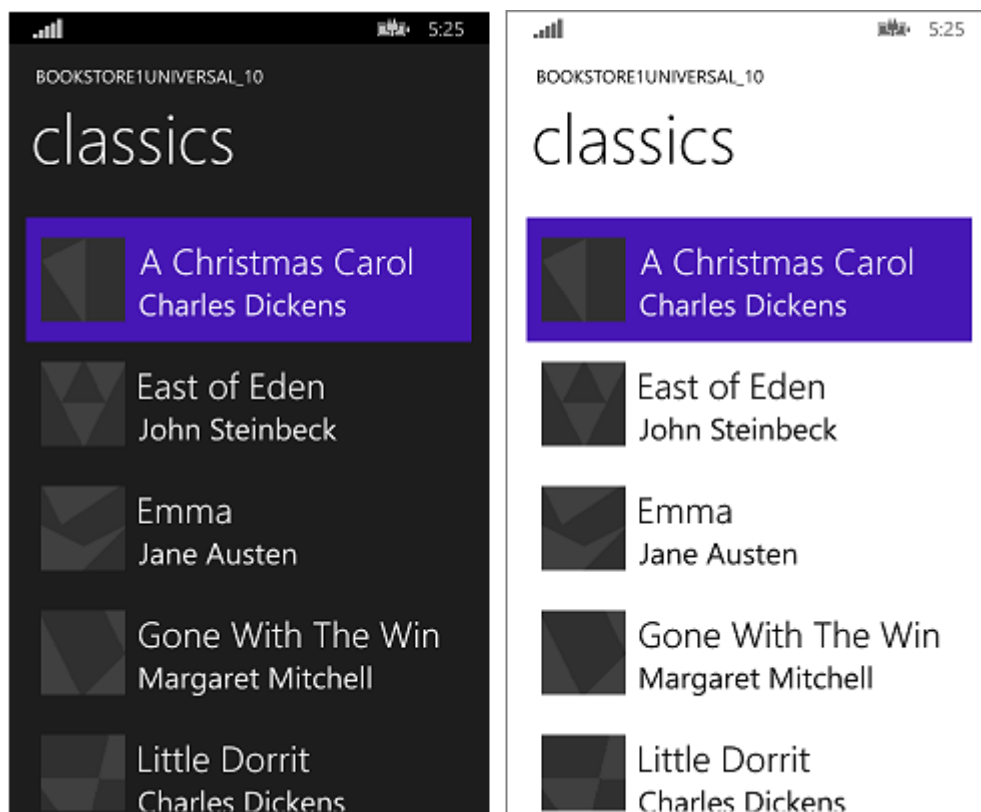
Universal styling

Now, we just need to make some final styling tweaks and confirm that the app looks good on desktop (and other) form factors as well as mobile. The steps are below. And you can use the links at the top of this topic to download the projects and see the results of all the changes between here and the end of the case study.

- To tighten up the spacing between items, find the `BookTemplate` data template in `MainPage.xaml` and delete the `Margin` attribute from the root `Grid`.
- If you want to give the page title a little more breathing room, you can reset the bottom margin of `-5.6` to `0` on the page title `TextBlock`.
- Now, we need to set `LayoutRoot`'s Background to the correct default value so that the app looks appropriate when running on all devices no matter what the theme is. Change it from `"Transparent"` to `"{ThemeResource ApplicationPageBackgroundThemeBrush}"`.

With a more sophisticated app, this would be the point at which we'd use the guidance in [Porting for form factor and user experience](#) and really make optimal use of the form factor of each of the many devices the app can now run on. But, for this simple app, we can stop here and see how the app looks after that last sequence of styling operations. It actually looks the same on mobile and desktop devices, although it's not making best use of space on wide form factors (but we'll investigate how to do that in a later case study).

See [Theme changes](#) to see how to control the theme of your app.



The ported Windows 10 app running on a Mobile device

An optional adjustment to the list box for Mobile devices

When the app is running on a Mobile device, the background of a list box is light by default in both themes. That may be the style that you prefer and, if so, then there's nothing more to do. But, controls are designed so that you can customize their look while leaving their behavior unaffected. So, if you want the list box to be dark in the dark theme—the way the original app looked—then follow [these instructions](#) under "An optional adjustment".

Conclusion

This case study showed the process of porting a very simple app—arguably an unrealistically simple one. For instance, list controls can be used for selection or for establishing a context for navigation; the app navigates to a page with more details about the item that was tapped. This particular app does nothing with the user's selection, and it has no navigation. Even so, the case study served to break the ice, to introduce the porting process, and to demonstrate important techniques that you can use in real UWP apps.

The next case study is [Bookstore2](#), in which we look at accessing and displaying grouped data.

Windows Phone Silverlight to UWP case study: Bookstore2

Article • 10/20/2022

This case study—which builds on the info given in [Bookstore1](#)—begins with a Windows Phone Silverlight app that displays grouped data in a **LongListSelector**. In the view model, each instance of the class **Author** represents the group of the books written by that author, and in the **LongListSelector**, we can either view the list of books grouped by author or we can zoom out to see a jump list of authors. The jump list affords much quicker navigation than scrolling through the list of books. We walk through the steps of porting the app to a Windows 10 Universal Windows Platform (UWP) app.

Note When opening Bookstore2Universal_10 in Visual Studio, if you see the message "Visual Studio update required", then follow the steps for setting Target Platform Version in [TargetPlatformVersion](#).

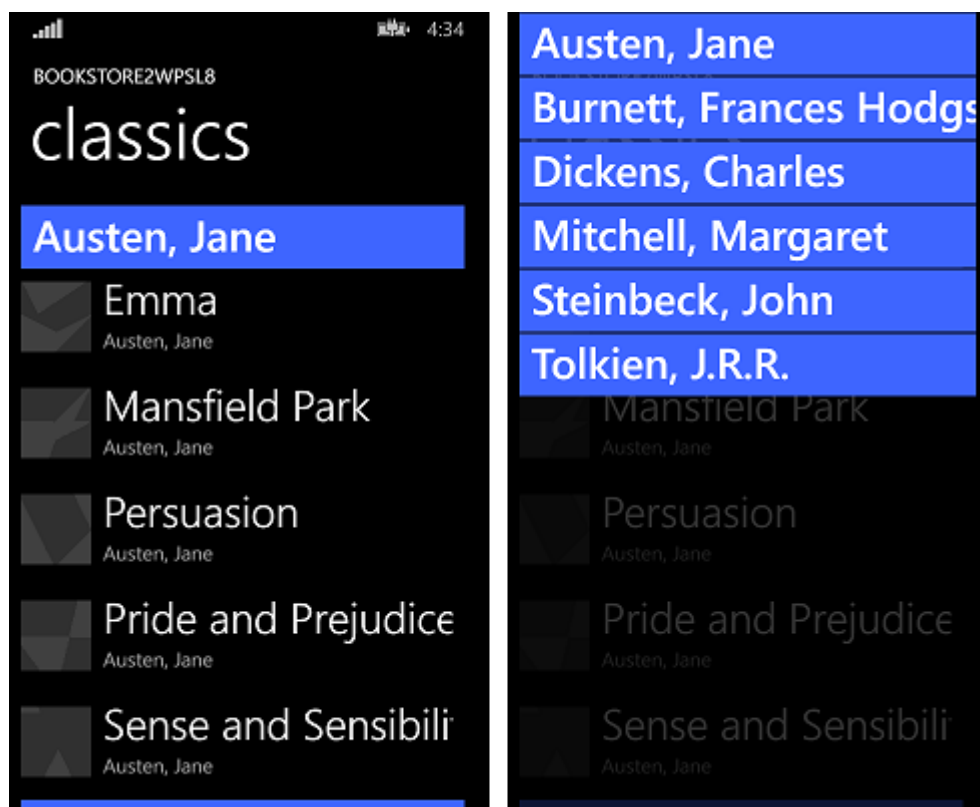
Downloads

[Download the Bookstore2WPSL8 Windows Phone Silverlight app](#) [↗](#).

[Download the Bookstore2Universal_10 Windows 10 app](#) [↗](#).

The Windows Phone Silverlight app

The illustration below shows what Bookstore2WPSL8—the app that we're going to port—looks like. It's a vertically-scrolling **LongListSelector** of books grouped by author. You can zoom out to the jump list, and from there, you can navigate back into any group. There are two main pieces to this app: the view model that provides the grouped data source, and the user interface that binds to that view model. As we'll see, both of these pieces port easily from Windows Phone Silverlight technology to the Universal Windows Platform (UWP).



Porting to a Windows 10 project

It's a quick task to create a new project in Visual Studio, copy files over to it from Bookstore2WPSL8, and include the copied files in the new project. Start by creating a new Blank Application (Windows Universal) project. Name it Bookstore2Universal_10. These are the files to copy over from Bookstore2WPSL8 to Bookstore2Universal_10.

- Copy the folder containing the book cover image PNG files (the folder is \Assets\CoverImages). After copying the folder, in **Solution Explorer**, make sure **Show All Files** is toggled on. Right-click the folder that you copied and click **Include In Project**. That command is what we mean by "including" files or folders in a project. Each time you copy a file or folder, click **Refresh** in **Solution Explorer** and then include the file or folder in the project. There's no need to do this for files that you're replacing in the destination.
- Copy the folder containing the view model source file (the folder is \ViewModel).
- Copy MainPage.xaml and replace the file in the destination.

We can keep the App.xaml, and App.xaml.cs that Visual Studio generated for us in the Windows 10 project.

Edit the source code and markup files that you just copied and change any references to the Bookstore2WPSL8 namespace to Bookstore2Universal_10. A quick way to do that is to use the **Replace In Files** feature. In the imperative code in the view model source file, these porting changes are needed.

- Change `System.ComponentModel.DesignerProperties` to `DesignMode` and then use the **Resolve** command on it. Delete the `IsInDesignTool` property and use IntelliSense to add the correct property name: `DesignModeEnabled`.
- Use the **Resolve** command on `ImageSource`.
- Use the **Resolve** command on `BitmapImage`.
- Delete `using System.Windows.Media;` and `using System.Windows.Media.Imaging;`.
- Change the value returned by the `Bookstore2Universal_10.BookstoreViewModel.AppName` property from "BOOKSTORE2WPSL8" to "BOOKSTORE2UNIVERSAL".
- Just as we did for [Bookstore1](#), update the implementation of the `BookSku.CoverImage` property (see [Binding an Image to a view model](#)).

In `MainPage.xaml`, these initial porting changes are needed.

- Change `phone:PhoneApplicationPage` to `Page` (including the occurrences in property element syntax).
- Delete the `phone` and `shell` namespace prefix declarations.
- Change "clr-namespace" to "using" in the remaining namespace prefix declaration.
- Delete `SupportedOrientations="Portrait"`, and `Orientation="Portrait"`, and configure **Portrait** in the app package manifest in the new project.
- Delete `shell:SystemTray.IsVisible="True"`.
- The types of the jump list item converters (which are present in the markup as resources) have moved to the [Windows.UI.Xaml.Controls.Primitives](#) namespace. So, add the namespace prefix declaration `Windows_UI_Xaml_Controls_Primitives` and map it to **Windows.UI.Xaml.Controls.Primitives**. On the jump list item converter resources, change the prefix from `phone:` to `Windows_UI_Xaml_Controls_Primitives:`.
- Just as we did for [Bookstore1](#), replace all references to the `PhoneTextExtraLargeStyle` **TextBlock** style with a reference to `SubtitleTextBlockStyle`, replace `PhoneTextSubtleStyle` with `SubtitleTextBlockStyle`, replace `PhoneTextNormalStyle` with `CaptionTextBlockStyle`, and replace `PhoneTextTitle1Style` with `HeaderTextBlockStyle`.
- There is one exception in `BookTemplate`. The style of the second **TextBlock** should reference `CaptionTextBlockStyle`.
- Remove the `FontFamily` attribute from the **TextBlock** inside `AuthorGroupHeaderTemplate` and set the `Background` of the **Border** to reference `SystemControlBackgroundAccentBrush` instead of `PhoneAccentBrush`.
- Because of [changes related to view pixels](#), go through the markup and multiply any fixed size dimension (margins, width, height, etc) by 0.8.

Replacing the LongListSelector

Replacing the **LongListSelector** with a **SemanticZoom** control will take several steps, so let's make a start on that. A **LongListSelector** binds directly to the grouped data source, but a **SemanticZoom** contains **ListView** or **GridView** controls, which bind indirectly to the data via a **CollectionViewSource** adapter. The **CollectionViewSource** needs to be present in the markup as a resource, so let's begin by adding that to the markup in **MainPage.xaml** inside `<Page.Resources>`.

XML

```
<CollectionViewSource
  x:Name="AuthorHasACollectionOfBookSku"
  Source="{Binding Authors}"
  IsSourceGrouped="true"/>
```

Note that the binding on **LongListSelector.ItemsSource** becomes the value of **CollectionViewSource.Source**, and **LongListSelector.IsGroupingEnabled** becomes **CollectionViewSource.IsSourceGrouped**. The **CollectionViewSource** has a name (note: not a key, as you might expect) so that we can bind to it.

Next, replace the `phone:LongListSelector` with this markup, which will give us a preliminary **SemanticZoom** to work with.

XML

```
<SemanticZoom>
  <SemanticZoom.ZoomedInView>
    <ListView
      ItemsSource="{Binding Source={StaticResource
        AuthorHasACollectionOfBookSku}}}"
      ItemTemplate="{StaticResource BookTemplate}">
      <ListView.GroupStyle>
        <GroupStyle
          HeaderTemplate="{StaticResource
            AuthorGroupHeaderTemplate}"
          HidesIfEmpty="True"/>
      </ListView.GroupStyle>
    </ListView>
  </SemanticZoom.ZoomedInView>
  <SemanticZoom.ZoomedOutView>
    <ListView
      ItemsSource="{Binding CollectionGroups, Source=
        {StaticResource AuthorHasACollectionOfBookSku}}}"
      ItemTemplate="{StaticResource ZoomedOutAuthorTemplate}">
    </SemanticZoom.ZoomedOutView>
  </SemanticZoom>
```

The **LongListSelector** notion of flat list and jump list modes is answered in the **SemanticZoom** notion of a zoomed-in and a zoomed-out view, respectively. The zoomed-in view is a property, and you set that property to an instance of a **ListView**. In this case, the zoomed-out view is also set to a **ListView**, and both **ListView** controls are bound to our **CollectionViewSource**. The zoomed-in view uses the same item template, group header template, and **HideEmptyGroups** setting (now named **HidesIfEmpty**) as the **LongListSelector**'s flat list does. And the zoomed-out view uses an item template very much like the one inside the **LongListSelector**'s jump list style (`AuthorNameJumpListStyle`). Also, note that the zoomed-out view binds to a special property of the **CollectionViewSource** named **CollectionGroups**, which is a collection containing the groups rather than the items.

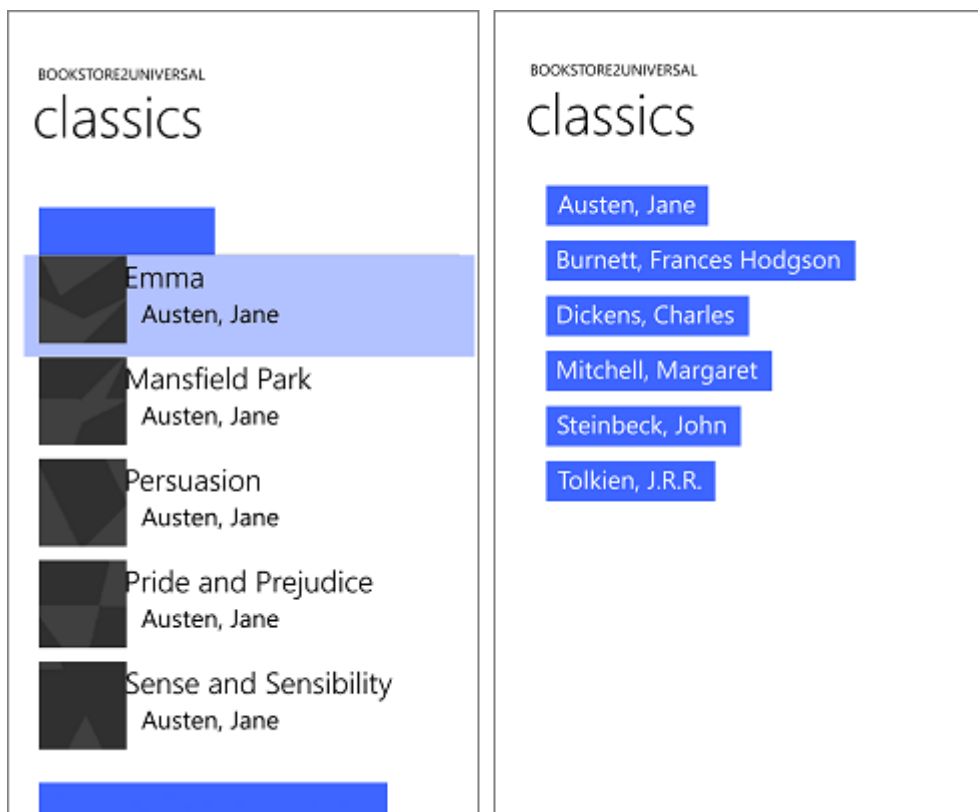
We no longer need `AuthorNameJumpListStyle`, at least not all of it. We only need the data template for the groups (which are authors in this app) in the zoomed-out view. So, we delete the `AuthorNameJumpListStyle` style and replace it with this data template.

XML

```
<DataTemplate x:Key="ZoomedOutAuthorTemplate">
    <Border Margin="9.6,0.8" Background="{Binding Converter=
{StaticResource JumpListItemBackgroundConverter}}">
        <TextBlock Margin="9.6,0,9.6,4.8" Text="{Binding Group.Name}"
Style="{StaticResource SubtitleTextBlockStyle}"
        Foreground="{Binding Converter={StaticResource
JumpListItemForegroundConverter}}" VerticalAlignment="Bottom"/>
    </Border>
</DataTemplate>
```

Note that, since the data context of this data template is a group rather than an item, we bind to a special property named **Group**.

You can build and run the app now. Here's how it looks on the mobile emulator.



The view model and the zoomed-in and zoomed-out views are working together correctly, although one issue is that we need to do a little more styling and templating work. For example, the correct styles and brushes are not yet being used, so the text is invisible on the group headers that you can click to zoom out. If you run the app on a desktop device, then you'll see a second issue, which is that the app doesn't yet adapt its user-interface to give the best experience and use of space on larger devices where windows can be potentially much larger than the screen of a mobile device. So, in the next few sections ([Initial styling and templating](#), [Adaptive UI](#), and [Final styling](#)), we'll remedy those issues.

Initial styling and templating

To space out the group headers nicely, edit `AuthorGroupHeaderTemplate` and set a **Margin** of `"0,0,0,9.6"` on the **Border**.

To space out the book items nicely, Edit `BookTemplate` and set the **Margin** to `"9.6,0"` on both **TextBlocks**.

To lay out the app name and the page title a little better, inside `TitlePanel`, remove the top **Margin** on the second **TextBlock** by setting the value to `"7.2,0,0,0"`. And on `TitlePanel` itself, set the margin to `0` (or whatever value looks good to you)

Change `LayoutRoot`'s Background to `"{ThemeResource ApplicationPageBackgroundThemeBrush}"`.

Adaptive UI

Because we started out with a phone app, it's no surprise that our ported app's UI layout really only makes sense for small devices and narrow windows at this stage in the process. But, we'd really like the UI layout to adapt itself and make better use of space when the app is running in a wide window (which is only possible on a device with a large screen), and for it only to use the UI that we have currently when the app's window is narrow (which happens on a small device, and can also happen on a large device).

We can use the adaptive Visual State Manager feature to achieve this. We'll set properties on visual elements so that, by default, the UI is laid out in the narrow state using the templates that we're using right now. Then, we'll detect when the app's window is wider-than-or-equal-to a specific size (measured in units of [effective pixels](#)), and in response, we'll change the properties of visual elements so that we get a larger, and wider, layout. We'll put those property changes in a visual state, and we'll use an adaptive trigger to continuously monitor and determine whether to apply that visual state, or not, depending on the width of the window in effective pixels. We're triggering on window width in this case, but it's possible to trigger on window height, too.

A minimum window width of 548 epx is appropriate for this use case because that's the size of the smallest device we would want to show the wide layout on. Phones are typically smaller than 548 epx, so on a small device like that, we'd remain in the default narrow layout. On a PC, the window will launch by default wide enough to trigger the switch to the wide state, which will display 250x250-sized items. From there, you'll be able to drag the window narrow enough to display a minimum of two columns of the 250x250 items. Any narrower than that and the trigger will deactivate, the wide visual state will be removed, and the default narrow layout will be in effect.

Before tackling the adaptive Visual State Manager piece, we first need to design the wide state and that means adding some new visual elements and templates to our markup. These steps describe how to do that. By way of naming conventions for visual elements and templates, we'll include the word "wide" in the name of any element or template that is for the wide state. If an element or template does not contain the word "wide", then you can assume that it is for the narrow state, which is the default state and whose property values are set as local values on visual elements in the page. Only the property values for the wide state are set via an actual Visual State in the markup.

- Make a copy of the [SemanticZoom](#) control in the markup and set `x:Name="narrowSeZo"` on the copy. On the original, set `x:Name="wideSeZo"` and also set `Visibility="Collapsed"` so that the wide one is not visible by default.
- In `wideSeZo`, change the **ListViews** to **GridViews** in both the zoomed-in view and the zoomed-out view.

- Make a copy of these three resources `AuthorGroupHeaderTemplate`, `ZoomedOutAuthorTemplate`, and `BookTemplate` and append the word `Wide` to the keys of the copies. Also, update `wideSeZo` so that it references the keys of these new resources.
- Replace the contents of `AuthorGroupHeaderTemplateWide` with `<TextBlock Style="{StaticResource SubheaderTextBlockStyle}" Text="{Binding Name}"/>`.
- Replace the contents of `ZoomedOutAuthorTemplateWide` with:

XML

```
<Grid HorizontalAlignment="Left" Width="250" Height="250" >
    <Border Background="{StaticResource
ListViewItemPlaceholderBackgroundThemeBrush}"/>
    <StackPanel VerticalAlignment="Bottom" Background="{StaticResource
ListViewItemOverlayBackgroundThemeBrush}">
        <TextBlock Foreground="{StaticResource
ListViewItemOverlayForegroundThemeBrush}"
            Style="{StaticResource SubtitleTextBlockStyle}"
            Height="80" Margin="15,0" Text="{Binding Group.Name}"/>
    </StackPanel>
</Grid>
```

- Replace the contents of `BookTemplateWide` with:

XML

```
<Grid HorizontalAlignment="Left" Width="250" Height="250">
    <Border Background="{StaticResource
ListViewItemPlaceholderBackgroundThemeBrush}"/>
    <Image Source="{Binding CoverImage}" Stretch="UniformToFill"/>
    <StackPanel VerticalAlignment="Bottom" Background="{StaticResource
ListViewItemOverlayBackgroundThemeBrush}">
        <TextBlock Style="{StaticResource SubtitleTextBlockStyle}"
            Foreground="{StaticResource
ListViewItemOverlaySecondaryForegroundThemeBrush}"
            TextWrapping="NoWrap" TextTrimming="CharacterEllipsis"
            Margin="12,0,24,0" Text="{Binding Title}"/>
        <TextBlock Style="{StaticResource CaptionTextBlockStyle}" Text=
"{Binding Author.Name}"
            Foreground="{StaticResource
ListViewItemOverlaySecondaryForegroundThemeBrush}" TextWrapping="NoWrap"
            TextTrimming="CharacterEllipsis" Margin="12,0,12,12"/>
    </StackPanel>
</Grid>
```

- For the wide state, the groups in the zoomed-in view will need more vertical breathing space around them. Creating and referencing an items panel template will give us the results we want. Here's how the markup looks.

XML

```

<ItemsPanelTemplate x:Key="ZoomedInItemsPanelTemplate">
    <ItemsWrapGrid Orientation="Horizontal" GroupPadding="0,0,0,20"/>
</ItemsPanelTemplate>
...

<SemanticZoom x:Name="wideSeZo" ... >
    <SemanticZoom.ZoomedInView>
        <GridView
            ...
            ItemsPanel="{StaticResource ZoomedInItemsPanelTemplate}">
            ...

```

- Finally, add the appropriate Visual State Manager markup as the first child of `LayoutRoot`.

XML

```

<Grid x:Name="LayoutRoot" ... >
    <VisualStateManager.VisualStateGroups>
        <VisualStateGroup>
            <VisualState x:Name="WideState">
                <VisualState.StateTriggers>
                    <AdaptiveTrigger MinWindowWidth="548"/>
                </VisualState.StateTriggers>
                <VisualState.Setters>
                    <Setter Target="wideSeZo.Visibility"
Value="Visible"/>
                    <Setter Target="narrowSeZo.Visibility"
Value="Collapsed"/>
                </VisualState.Setters>
            </VisualState>
        </VisualStateGroup>
    </VisualStateManager.VisualStateGroups>
    ...

```

Final styling

All that remains are some final styling tweaks.

- In `AuthorGroupHeaderTemplate`, set `Foreground="White"` on the **TextBlock** so that it looks correct when running on the mobile device family.
- Add `FontWeight="SemiBold"` to the **TextBlock** in both `AuthorGroupHeaderTemplate` and `ZoomedOutAuthorTemplate`.

- In `narrowSeZo`, the group headers and the authors in the zoomed-out view are left-aligned instead of stretched, so let's work on that. We'll create a `HeaderContainerStyle` for the zoomed-in view with `HorizontalContentAlignment` set to `Stretch`. And we'll create an `ItemContainerStyle` for the zoomed-out view containing that same `Setter`. Here's what that looks like.

XML

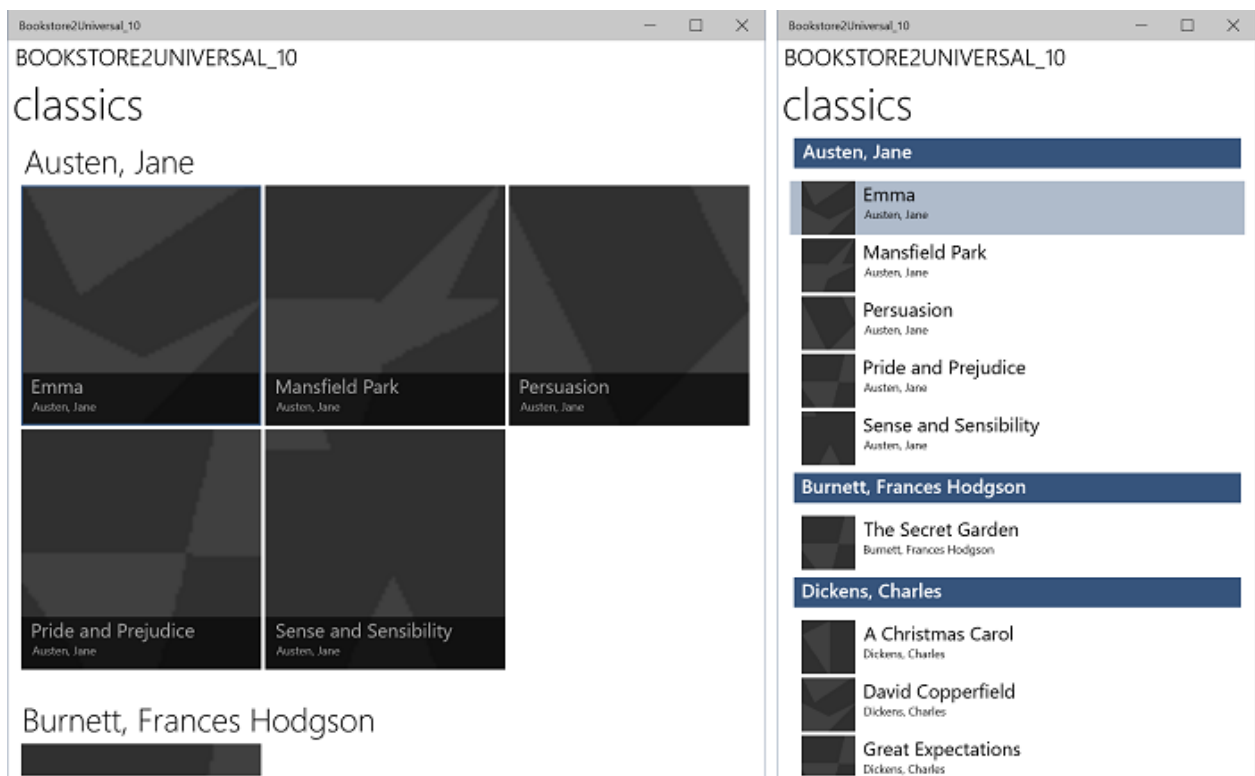
```
<Style x:Key="AuthorGroupHeaderContainerStyle"
TargetType="ListViewHeaderItem">
    <Setter Property="HorizontalContentAlignment" Value="Stretch"/>
</Style>

<Style x:Key="ZoomedOutAuthorItemContainerStyle"
TargetType="ListViewItem">
    <Setter Property="HorizontalContentAlignment" Value="Stretch"/>
</Style>

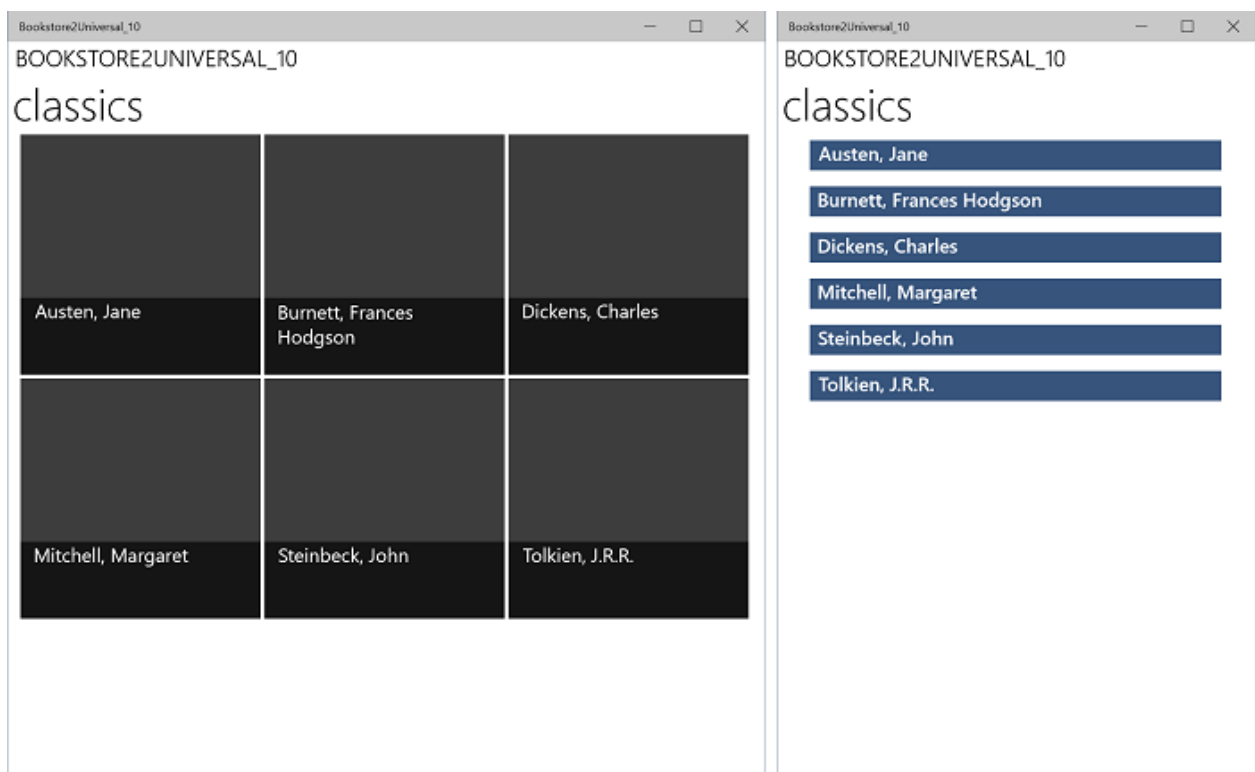
...

<SemanticZoom x:Name="narrowSeZo" ... >
    <SemanticZoom.ZoomedInView>
        <ListView
            ...
            <ListView.GroupStyle>
                <GroupStyle
                    ...
                    HeaderContainerStyle="{StaticResource
AuthorGroupHeaderContainerStyle}"
                    ...
                </GroupStyle>
            </ListView.GroupStyle>
        </ListView>
    </SemanticZoom.ZoomedInView>
    <SemanticZoom.ZoomedOutView>
        <ListView
            ...
            ItemContainerStyle="{StaticResource
ZoomedOutAuthorItemContainerStyle}"
            ...
        </ListView>
    </SemanticZoom.ZoomedOutView>
</SemanticZoom>
```

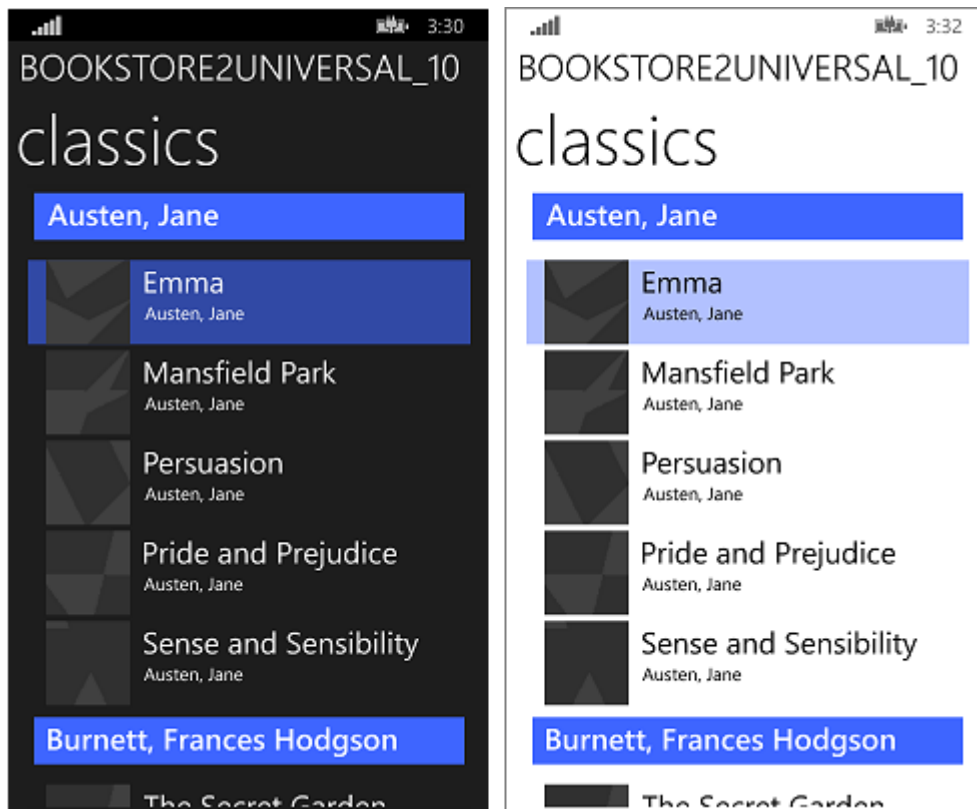
That last sequence of styling operations leaves the app looking like this.



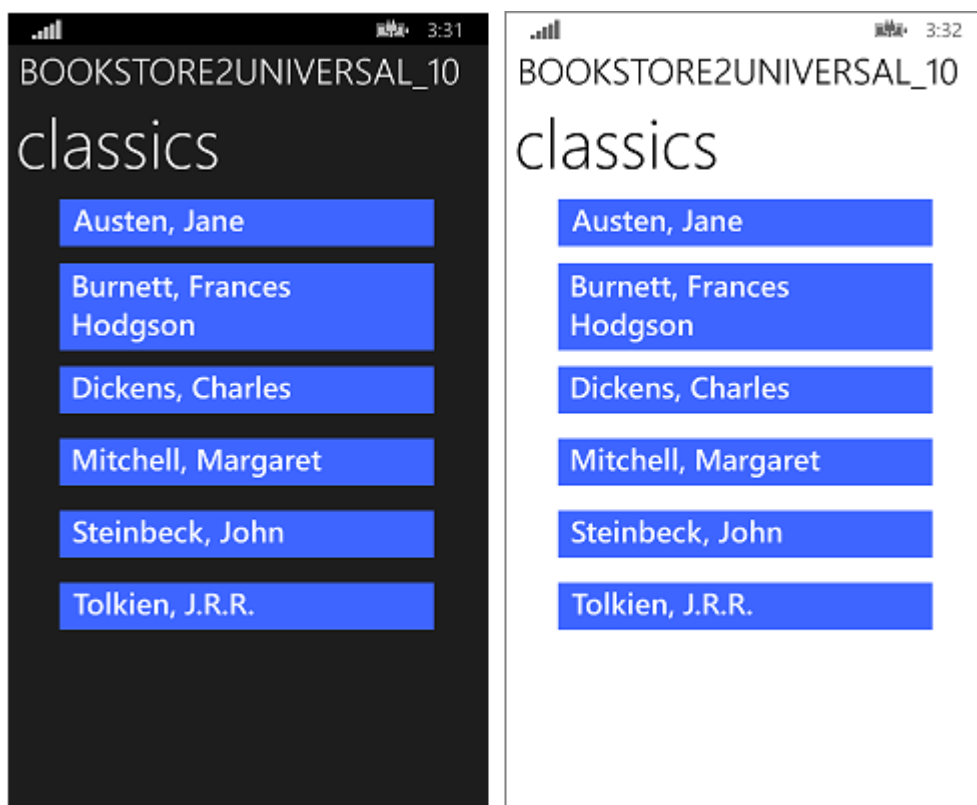
The ported Windows 10 app running on a Desktop device, zoomed-in view, two sizes of window



The ported Windows 10 app running on a Desktop device, zoomed-out view, two sizes of window



The ported Windows 10 app running on a Mobile device, zoomed-in view



The ported Windows 10 app running on a Mobile device, zoomed-out view

Making the view model more flexible

This section contains an example of facilities that open up to us by virtue of having moved our app to use the UWP. Here, we explain optional steps that you can follow to make your view model more flexible when accessed via a **CollectionViewSource**. The view model (the source file is in ViewModel\BookstoreViewModel.cs) that we ported from the Windows Phone Silverlight app Bookstore2WPSL8 contains a class named Author, which derives from **List<T>**, where T is BookSku. That means that the Author class *is a* group of BookSku.

When we bind **CollectionViewSource.Source** to Authors, the only thing we're communicating is that each Author in Authors is a group of *something*. We leave it to the **CollectionViewSource** to determine that Author is, in this case, a group of BookSku. That works: but it's not flexible. What if we want Author to be *both* a group of BookSku *and* a group of the addresses where the author has lived? Author can't *be* both of those groups. But, Author can *have* any number of groups. And that's the solution: use the *has-a-group* pattern instead of, or in addition to, the *is-a-group* pattern that we're using currently. Here's how:

- Change Author so that it no longer derives from **List<T>**.
- Add this field to
- Add this property to
- And of course we can repeat the above two steps to add as many groups to Author as we need.
- Change the implementation of the AddBookSku method to `this.BookSkus.Add(bookSku);`.
- Now that Author *has* at least one group, we need to communicate to the **CollectionViewSource** which of those groups it should use. To do that, add this property to the **CollectionViewSource**: `ItemsPath="BookSkus"`

Those changes leave this app functionally unchanged, but you now know how you could extend Author, and the **CollectionViewSource**, should you need to. Let's make one last change to Author so that, if we use it *without* specifying **CollectionViewSource.ItemsPath**, a default group of our choosing will be used:

C#

```
public class Author : IEnumerable<BookSku>
{
    ...

    public IEnumerator<BookSku> GetEnumerator()
    {
        return this.BookSkus.GetEnumerator();
    }
    System.Collections.IEnumerator
```

```
System.Collections.IEnumerable.GetEnumerator()  
{  
    return this.BookSkus.GetEnumerator();  
}
```

And now we can choose to remove `ItemsPath="BookSkus"` if we like and the app will still behave the same way.

Conclusion

This case study involved a more ambitious user interface than the previous one. All of the facilities and concepts of the Windows Phone Silverlight **LongListSelector**—and more—were found to be available to a UWP app in the form of **SemanticZoom**, **ListView**, **GridView**, and **CollectionViewSource**. We showed how to re-use, or copy-and-edit, both imperative code and markup in a UWP app to achieve functionality, UI, and interactions tailored to suit the narrowest and widest Windows device form factors and all sizes in-between.

Turn your website into a high quality PWA

Article • 07/23/2024

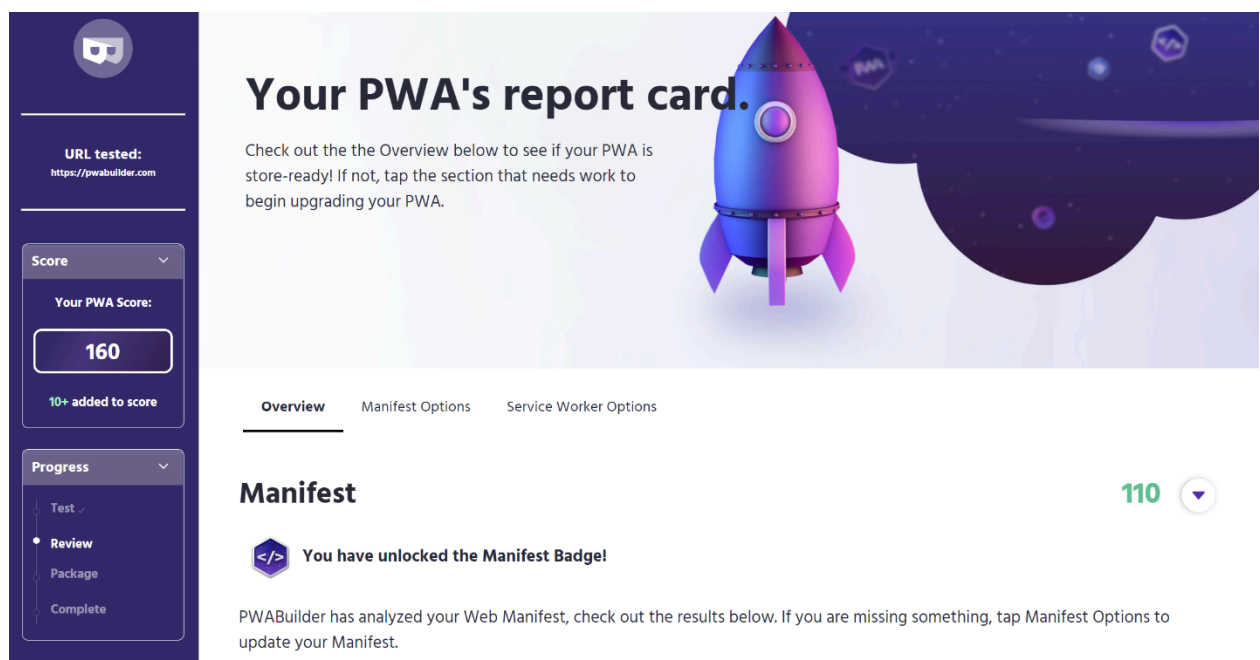
Turning your website into a fully functional high quality PWA (Progressive Web Application) can be done in a lunch break! In this article we'll take you through the end-to-end process.

Step 1: Create a report card in PWA Builder

First, you'll want to check if your website has the features and metadata that great PWAs need.

1. Navigate to [PWA Builder](https://pwabuilder.com)
2. Enter the URL of the website you'd like to convert to a PWA and click **Start**
3. PWA builder will display a PWA Report Card that indicates the features your website has and does not have.

Step 2: Review your PWA Report Card



Your PWA Report Card contains your website's score, out of a maximum score of 100. Scores are based off of three categories of features.

Manifest

PWA evaluates your website's [Web Application Manifest](#) if one is available, and identifies fields or values that are required, recommended, and optional. You can improve your score by adding missing values, either by editing and re-publishing your manifest file, or by using the online manifest editor.

⚠ Warning

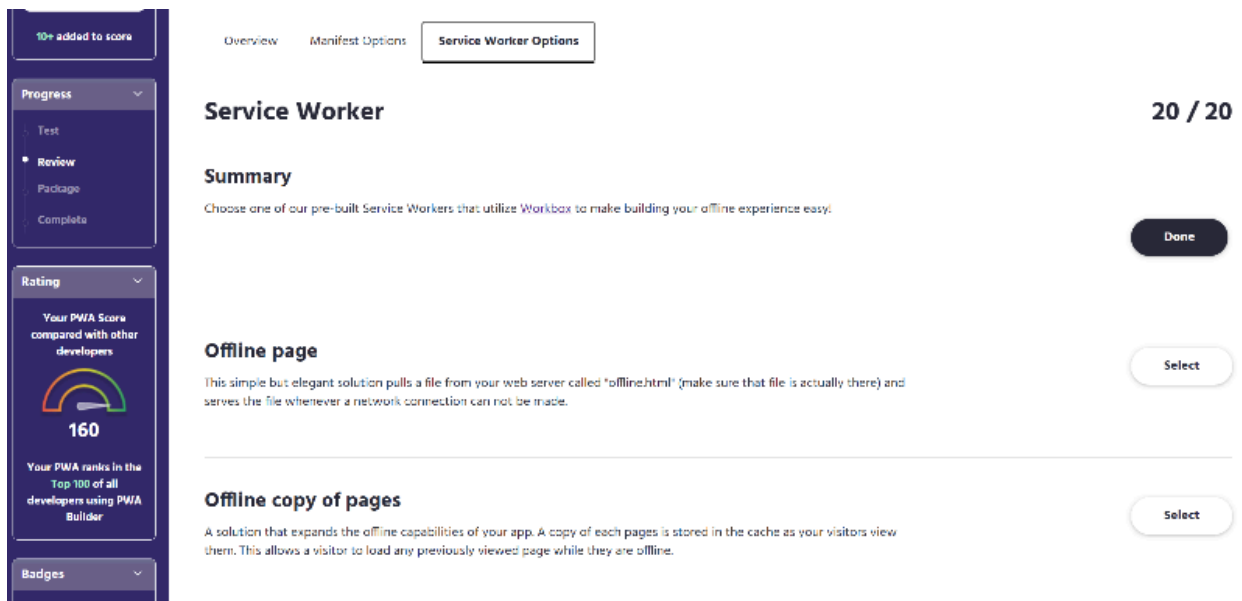
Changes made in the online manifest editor will not be published to your website. The changes you make will only be used by PWA Builder when creating your PWA.

Required values must be present, otherwise PWA builder won't be able to create your PWA. Required fields include simply having a manifest, icons, name, short name, and a start_url.

Recommended fields aren't required, but are important for your PWA's user experience. It is strongly recommended you provide all recommended values, though PWA Builder can create a PWA without them. Recommended values include display mode, splash screen background color, description, orientation, screenshots, high resolution icon, maskable icon, categories, and shortcuts.

Optional fields are not required, but can be specified if appropriate. Optional fields include an age rating and related applications.

Service Worker



The screenshot displays the PWA Builder interface. On the left, a sidebar shows a progress bar with steps: Test, Review, Package, and Complete. Below the progress bar, a 'Rating' section shows a score of 160, indicating the PWA ranks in the top 100 of all developers using PWA Builder. The main content area is titled 'Service Worker' and has a '20 / 20' status. It includes a 'Summary' section with a 'Done' button. Below the summary, there are two sections: 'Offline page' and 'Offline copy of pages', each with a 'Select' button. The 'Offline page' section describes a solution that pulls a file from the web server called 'offline.html' and serves it when a network connection cannot be made. The 'Offline copy of pages' section describes a solution that expands the offline capabilities of the app by storing a copy of each page in the cache as visitors view them.

A service worker runs in the background to enable rich web app features where there would otherwise be 404 errors. PWA Builder requires a service worker to create your PWA, but if you do not have one you can use one of several prebuilt service workers provided by PWA Builder. To use a prebuilt service worker:

1. Choose the **Service Worker Options** tab in the menu.
2. Select the appropriate service worker from the curated list. Note that there are many prebuilt options; make sure to pick the service worker most appropriate for your specific use case.
3. Once you've selected the service worker you'd like to use, you'll be returned to your PWA Report Card.

Security

All PWAs built with PWA Builder require three security protocols.

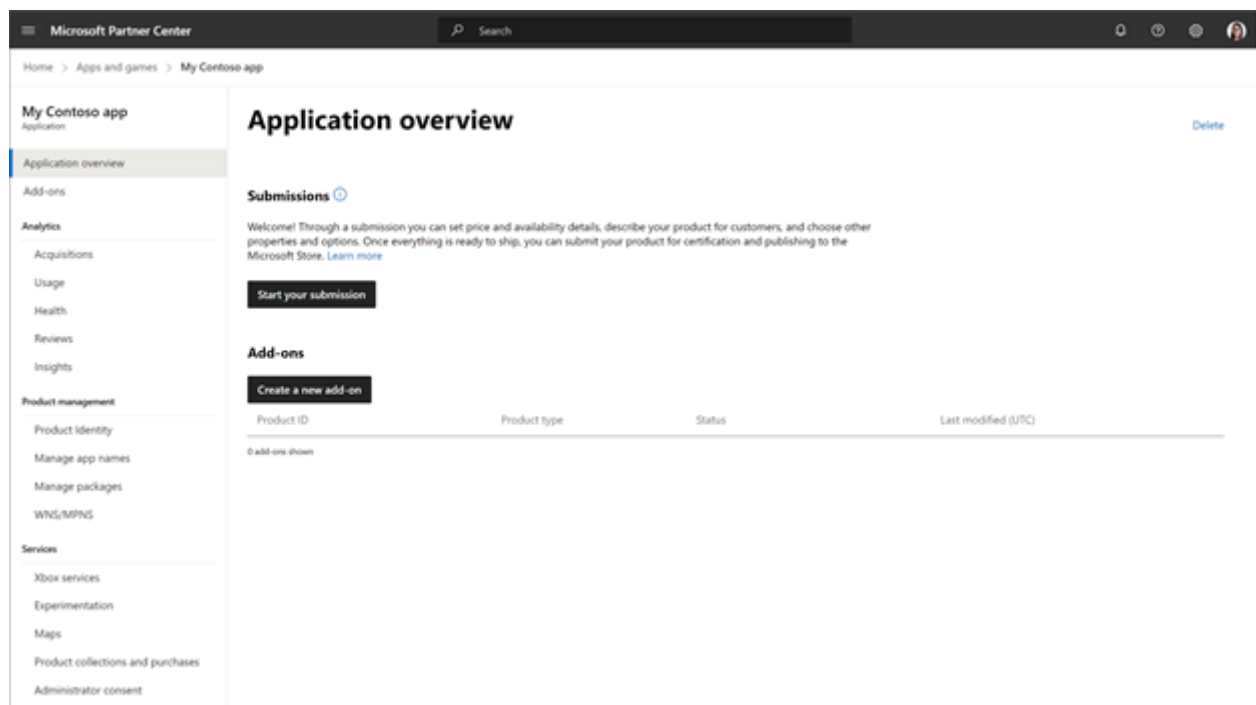
1. Your site must use HTTPS.
2. Your site must have a corresponding HTTPS certificate.

If your website does not have a HTTPS certificate present you can publish to Azure to get built-in HTTPS support. Alternatively there are free third-party tools such as [Letsencrypt](#) that will allow you to obtain a HTTPS certificate for free.

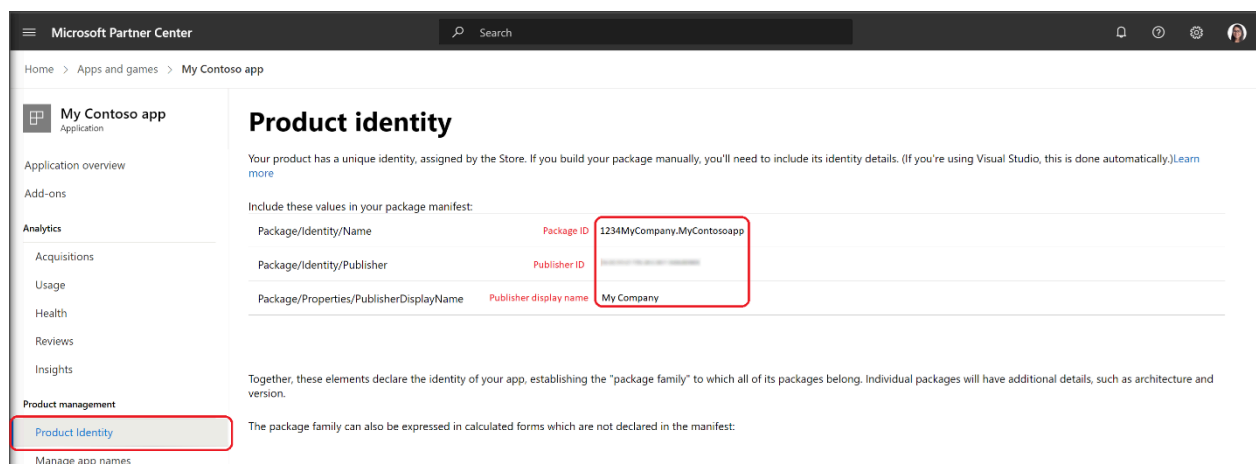
3. Your site must not contain mixed content. Mixed content is when a page served over HTTPS contains resources loaded over HTTP. Mixed content compromises the security of HTTPS and your PWA.

Step 3: Collect important information from Partner Center

You'll need several pieces of information from your Partner Center account to create your PWA. If you do not have an MSA, [click here for instructions on how to get started](#). You will also need to enroll in the Windows Developer Program.



If you have not already done so, you'll need to [create your PWA app by reserving a new name](#). Once you've reserved your name, click the **Start your submission** button to create a new app submission.



Next, click on the **Product Management** tab and select **Product Identity**. The product identity page will list your product's package ID, your publisher ID, and your publisher display name. Save these values and return to PWA Builder.

Step 4: Generate your PWA on PWA Builder



Windows

Publish your PWA to the Microsoft Store to make it available to the 1 billion Windows users worldwide.

Generate

Test Package

You now have everything you need to create your PWA on PWA Builder. Return to the PWA Builder site and click **Generate**.



Microsoft Store Options

Customize your Windows package below!

Package ID ?

Publisher display name ?

Publisher ID ?

All Settings ▶

Your download will contain instructions for submitting your app to the Microsoft Store.

Your app will be powered by Chromium-based Edge platform (preview).

Generate

PWA builder will prompt you for the information you obtained from Partner Center in step 3. Fill in the values and click **Generate**.

 utils	File folder			
 install	Windows PowerShell Script	1 KB	No	2 KB
 PWABuilder	MSIXBUNDLE File	1,296 KB	No	1,507 KB
 PWABuilder.classic	APPXBUNDLE File	1,294 KB	No	1,552 KB
 PWABuilder.sideload	MSIX File	1,295 KB	No	1,505 KB
 readme	Microsoft Edge HTML Docum...	1 KB	No	1 KB

Once PWA Builder has created your PWA, your browser will automatically download it. Your PWA is packaged in a .zip file that contains six files.

The **msix** file is your PWA's main app package. This file will install your PWA on the user's machine.

The **appx** file is a classic app package. It is used to install your PWA on older versions of Windows. See [Classic app packages](#) for more information.

Step 5: Submit your app packages to the Microsoft Store

These packages can be submitted to the Microsoft Store in the same way any other app packaged as an MSIX file can be. For further instructions submitting your PWA packages to the store, see [App submissions](#).

Collaborate with us on GitHub

The source for this content can be found on GitHub, where you can also create and review issues and pull requests. For more information, see [our contributor guide](#).



Windows developer feedback

Windows developer is an open source project. Select a link to provide feedback:



[Open a documentation issue](#)



[Provide product feedback](#)

Threading and async programming

Article • 10/20/2022

Threading and async programming enables your app to accomplish work asynchronously in parallel threads.

Your app can use the thread pool to accomplish work asynchronously in parallel threads. The thread pool manages a set of threads and uses a queue to assign work items to threads as they become available. The thread pool is similar to the asynchronous programming patterns available in the Windows Runtime because it can be used to accomplish extended work without blocking the UI, but the thread pool offers more control than the asynchronous programming patterns and you can use it to complete multiple work items in parallel. You can use the thread pool to:

- Submit work items, control their priority, and cancel work items.
- Schedule work items using timers and periodic timers.
- Set aside resources for critical work items.
- Run work items in response to named events and semaphores.

The thread pool is more efficient at managing threads because it reduces the overhead of creating and destroying threads. This means it has access to optimize threads across multiple CPU cores, and it can balance thread resources between apps and when background tasks are running. Using the built-in thread pool is convenient because you focus on writing code that accomplishes a task instead of the mechanics of thread management.

Topic	Description
Asynchronous programming (UWP apps)	This topic describes asynchronous programming in the Universal Windows Platform (UWP) and its representation in C#, Microsoft Visual Basic .NET, Visual C++ component extensions (C++/CX), and JavaScript.
Asynchronous programming in C++/CX (UWP apps)	This article describes the recommended way to consume asynchronous methods in C++/CX by using the <code>task</code> class that's defined in the <code>concurrency</code> namespace in <code>ppltasks.h</code> .
Best practices for using the thread pool	This topic describes best practices for working with the thread pool.

Topic	Description
Call asynchronous APIs in C# or Visual Basic	The Universal Windows Platform (UWP) includes many asynchronous APIs to ensure that your app remains responsive when it does work that might take an extended amount of time. This topic discusses how to use asynchronous methods from the UWP in C# or Microsoft Visual Basic.
Create a periodic work item	Learn how to create a work item that repeats periodically.
Submit a work item to the thread pool	Learn how to do work in a separate thread by submitting a work item to the thread pool.
Use a timer to submit a work item	Learn how to create a work item that runs after a timer elapses.

Asynchronous programming

Article • 10/20/2022

This topic describes asynchronous programming in the Universal Windows Platform (UWP) and its representation in C#, Microsoft Visual Basic .NET, C++, and JavaScript.

Using asynchronous programming helps your app stay responsive when it does work that might take an extended amount of time. For example, an app that downloads content from the Internet might spend several seconds waiting for the content to arrive. If you used a synchronous method on the UI thread to retrieve the content, the app is blocked until the method returns. The app won't respond to user interaction, and because it seems non-responsive, the user might become frustrated. A much better way is to use asynchronous programming, where the app continues to run and respond to the UI while it waits for an operation to complete.

For methods that might take a long time to complete, asynchronous programming is the norm and not the exception in the UWP. JavaScript, C#, Visual Basic, and C++ each provide language support for asynchronous methods.

Asynchronous programming in the UWP

Many UWP features, such as the [MediaCapture](#) APIs and [StorageFile](#) APIs, are exposed as asynchronous APIs. By convention, the names of asynchronous APIs end with "Async" to indicate that part of their execution is likely to take place after control has returned to the caller.

When you use asynchronous APIs in your Universal Windows Platform (UWP) app, your code makes non-blocking calls in a consistent way. When you implement these asynchronous patterns in your own API definitions, callers can understand and use your code in a predictable way.

Here are some common tasks that require calling asynchronous Windows Runtime APIs.

- Displaying a message dialog
- Working with the file system, displaying a file picker
- Sending and receiving data to and from the Internet
- Using sockets, streams, connectivity
- Working with appointments, contacts, calendar

- Working with file types, such as opening Portable Document Format (PDF) files or decoding image or media formats
- Interacting with a device or a service

With UWP asynchronous pattern, you may be able to avoid explicitly manage threads at all. Each programming language supports the asynchronous pattern for the UWP in its own way:

Programming language	Asynchronous representation
C#	<code>async</code> keyword, <code>await</code> operator
Visual Basic	<code>Async</code> keyword, <code>Await</code> operator
C++/WinRT	coroutine, and <code>co_await</code> operator
C++/CX	<code>task</code> class, <code>.then</code> method
JavaScript	promise object, <code>then</code> function

Asynchronous patterns in UWP using C# and Visual Basic

A typical segment of code written in C# or Visual Basic executes synchronously, meaning that when a line executes, it finishes before the next line executes. There have been previous Microsoft .NET programming models for asynchronous execution, but the resulting code tends to emphasize the mechanics of executing asynchronous code instead of focusing on the task that the code is trying to accomplish. The UWP, .NET framework, and C# and Visual Basic compilers have added features that abstract the asynchronous mechanics out of your code. For .NET and the UWP you can write asynchronous code that focuses on what your code does instead of how and when to do it. Your asynchronous code will look reasonably similar to synchronous code. For more info, see [Call asynchronous APIs in C# or Visual Basic](#).

Asynchronous patterns in UWP with C++/WinRT

With C++/WinRT, you use coroutines, and the `co_await` operator. For more info, and code examples, see [Asynchronous programming in C++/WinRT](#).

Asynchronous patterns in UWP with C++/CX

In C++/CX, asynchronous programming is based on the [task class](#), and its [then method](#). The syntax is similar to that of JavaScript promises. The **task class** and its related types also provide the capability for cancellation and management of the thread context. For more info, see [Asynchronous programming in C++/CX](#).

The [create_async function](#) provides support for producing asynchronous APIs that can be consumed from JavaScript or any other language that supports the UWP. For more info, see [Creating Asynchronous Operations in C++/CX](#).

Asynchronous patterns in UWP using JavaScript

In JavaScript, asynchronous programming follows the [Common JS Promises/A](#)  proposed standard by having asynchronous methods return promise objects. Promises are used in both the UWP and Windows Library for JavaScript.

A promise object represents a value that will be fulfilled in the future. In the UWP you get a promise object from a factory function, which by convention has a name that ends with "Async".

In many cases, calling an asynchronous function is almost as simple as calling a conventional function. The difference is that you use the [then](#) or the [done](#) method to assign the handlers for results or errors and to start the operation.

Related topics

- [Call asynchronous APIs in C# or Visual Basic](#)
- [Asynchronous Programming with Async and Await \(C# and Visual Basic\)](#)
- [Reversi sample feature scenarios: asynchronous code](#)

Asynchronous programming in C++/CX

Article • 10/20/2022

ⓘ Note

This topic exists to help you maintain your C++/CX application. But we recommend that you use C++/WinRT for new applications. C++/WinRT is an entirely standard modern C++17 language projection for Windows Runtime (WinRT) APIs, implemented as a header-file-based library, and designed to provide you with first-class access to the modern Windows API.

This article describes the recommended way to consume asynchronous methods in Visual C++ component extensions (C++/CX) by using the `task` class that's defined in the `concurrency` namespace in `ppltasks.h`.

Windows Runtime asynchronous types

The Windows Runtime features a well-defined model for calling asynchronous methods and provides the types that you need to consume such methods. If you are not familiar with the Windows Runtime asynchronous model, read [Asynchronous Programming](#) before you read the rest of this article.

Although you can consume the asynchronous Windows Runtime APIs directly in C++, the preferred approach is to use the [task class](#) and its related types and functions, which are contained in the [concurrency](#) namespace and defined in `<ppltasks.h>`. The `concurrency::task` is a general-purpose type, but when the `/ZW` compiler switch—which is required for Universal Windows Platform (UWP) apps and components—is used, the `task` class encapsulates the Windows Runtime asynchronous types so that it's easier to:

- chain multiple asynchronous and synchronous operations together
- handle exceptions in task chains
- perform cancellation in task chains
- ensure that individual tasks run in the appropriate thread context or apartment

This article provides basic guidance about how to use the `task` class with the Windows Runtime asynchronous APIs. For more complete documentation about `task` and its related methods including [create_task](#), see [Task Parallelism \(Concurrency Runtime\)](#).

Consuming an async operation by using a task

The following example shows how to use the task class to consume an `async` method that returns an `IAsyncOperation` interface and whose operation produces a value. Here are the basic steps:

1. Call the `create_task` method and pass it the `IAsyncOperation^` object.
2. Call the member function `task::then` on the task and supply a lambda that will be invoked when the asynchronous operation completes.

C++

```
#include <ppltasks.h>
using namespace concurrency;
using namespace Windows::Devices::Enumeration;
...
void App::TestAsync()
{
    //Call the *Async method that starts the operation.
    IAsyncOperation<DeviceInformationCollection^> deviceOp =
        DeviceInformation::FindAllAsync();

    // Explicit construction. (Not recommended)
    // Pass the IAsyncOperation to a task constructor.
    // task<DeviceInformationCollection^> deviceEnumTask(deviceOp);

    // Recommended:
    auto deviceEnumTask = create_task(deviceOp);

    // Call the task's .then member function, and provide
    // the lambda to be invoked when the async operation completes.
    deviceEnumTask.then( [this] (DeviceInformationCollection^ devices )
    {
        for(int i = 0; i < devices->Size; i++)
        {
            DeviceInformation^ di = devices->GetAt(i);
            // Do something with di...
        }
    }); // end lambda
    // Continue doing work or return...
}
```

The task that's created and returned by the `task::then` function is known as a *continuation*. The input argument (in this case) to the user-provided lambda is the result that the task operation produces when it completes. It's the same value that would be retrieved by calling `IAsyncOperation::GetResults` if you were using the `IAsyncOperation` interface directly.

The `task::then` method returns immediately, and its delegate doesn't run until the asynchronous work completes successfully. In this example, if the asynchronous operation causes an exception to be thrown, or ends in the canceled state as a result of a cancellation request, the continuation will never execute. Later, we'll describe how to write continuations that execute even if the previous task was cancelled or failed.

Although you declare the task variable on the local stack, it manages its lifetime so that it is not deleted until all of its operations complete and all references to it go out of scope, even if the method returns before the operations complete.

Creating a chain of tasks

In asynchronous programming, it's common to define a sequence of operations, also known as *task chains*, in which each continuation executes only when the previous one completes. In some cases, the previous (or *antecedent*) task produces a value that the continuation accepts as input. By using the `task::then` method, you can create task chains in an intuitive and straightforward manner; the method returns a `task<T>` where `T` is the return type of the lambda function. You can compose multiple continuations into a task chain: `myTask.then(...).then(...).then(...);`

Task chains are especially useful when a continuation creates a new asynchronous operation; such a task is known as an asynchronous task. The following example illustrates a task chain that has two continuations. The initial task acquires the handle to an existing file, and when that operation completes, the first continuation starts up a new asynchronous operation to delete the file. When that operation completes, the second continuation runs, and outputs a confirmation message.

C++

```
#include <ppltasks.h>
using namespace concurrency;
...
void App::DeleteWithTasks(String^ fileName)
{
    using namespace Windows::Storage;
    StorageFolder^ localFolder = ApplicationData::Current->LocalFolder;
    auto getFileTask = create_task(localFolder->GetFileAsync(fileName));

    getFileTask.then([](StorageFile^ storageFileSample) -> IAsyncAction^ {
        return storageFileSample->DeleteAsync();
    }).then([](void) {
        OutputDebugString(L"File deleted.");
    });
}
```

The previous example illustrates four important points:

- The first continuation converts the `IAsyncAction^` object to a `task<void>` and returns the `task`.
- The second continuation performs no error handling, and therefore takes `void` and not `task<void>` as input. It is a value-based continuation.
- The second continuation doesn't execute until the `DeleteAsync` operation completes.
- Because the second continuation is value-based, if the operation that was started by the call to `DeleteAsync` throws an exception, the second continuation doesn't execute at all.

Note Creating a task chain is just one of the ways to use the `task` class to compose asynchronous operations. You can also compose operations by using join and choice operators `&&` and `||`. For more information, see [Task Parallelism \(Concurrency Runtime\)](#).

Lambda function return types and task return types

In a task continuation, the return type of the lambda function is wrapped in a `task` object. If the lambda returns a `double`, then the type of the continuation task is `task<double>`. However, the task object is designed so that it doesn't produce needlessly nested return types. If a lambda returns an `IAsyncOperation<SyndicationFeed^>^`, the continuation returns a `task<SyndicationFeed^>`, not a `task<task<SyndicationFeed^>>` or `task<IAsyncOperation<SyndicationFeed^>^>^`. This process is known as *asynchronous unwrapping* and it also ensures that the asynchronous operation inside the continuation completes before the next continuation is invoked.

In the previous example, notice that the task returns a `task<void>` even though its lambda returned an `IAsyncInfo` object. The following table summarizes the type conversions that occur between a lambda function and the enclosing task:

lambda return type	.then return type
<code>TResult</code>	<code>task<TResult></code>
<code>IAsyncOperation<TResult>^</code>	<code>task<TResult></code>
<code>IAsyncOperationWithProgress<TResult, TProgress>^</code>	<code>task<TResult></code>

lambda return type	<code>.then</code> return type
<code>IAsyncAction^</code>	<code>task<void></code>
<code>IAsyncActionWithProgress<TProgress>^</code>	<code>task<void></code>
<code>task<TResult></code>	<code>task<TResult></code>

Canceling tasks

It is often a good idea to give the user the option to cancel an asynchronous operation. And in some cases you might have to cancel an operation programmatically from outside the task chain. Although each **Async* return type has a **Cancel** method that it inherits from **IAsyncInfo**, it's awkward to expose it to outside methods. The preferred way to support cancellation in a task chain is to use a **cancellation_token_source** to create a **cancellation_token**, and then pass the token to the constructor of the initial task. If an asynchronous task is created with a cancellation token, and `[cancellation_token_source::cancel](/cpp/parallel/concrt/reference/cancellation-token-source-class?view=vs-2017& -view=true)` is called, the task automatically calls **Cancel** on the **IAsync*** operation and passes the cancellation request down its continuation chain. The following pseudocode demonstrates the basic approach.

C++

```
//Class member:
cancellation_token_source m_fileTaskTokenSource;

// Cancel button event handler:
m_fileTaskTokenSource.cancel();

// task chain
auto getFileTask2 = create_task(documentsFolder->GetFileAsync(fileName),
                               m_fileTaskTokenSource.get_token());
//getFileTask2.then ...
```

When a task is canceled, a **task_canceled** exception is propagated down the task chain. Value-based continuations will simply not execute, but task-based continuations will cause the exception to be thrown when **task::get** is called. If you have an error-handling continuation, make sure that it catches the **task_canceled** exception explicitly. (This exception is not derived from **Platform::Exception**.)

Cancellation is cooperative. If your continuation does some long-running work beyond just invoking a UWP method, then it is your responsibility to check the state of the cancellation token periodically and stop execution if it is canceled. After you clean up all

resources that were allocated in the continuation, call [cancel_current_task](#) to cancel that task and propagate the cancellation down to any value-based continuations that follow it. Here's another example: you can create a task chain that represents the result of a [FileSavePicker](#) operation. If the user chooses the **Cancel** button, the [IAsyncInfo::Cancel](#) method is not called. Instead, the operation succeeds but returns `nullptr`. The continuation can test the input parameter and call `cancel_current_task` if the input is `nullptr`.

For more information, see [Cancellation in the PPL](#)

Handling errors in a task chain

If you want a continuation to execute even if the antecedent was canceled or threw an exception, then make the continuation a task-based continuation by specifying the input to its lambda function as a `task<TResult>` or `task<void>` if the lambda of the antecedent task returns an [IAsyncAction^](#).

To handle errors and cancellation in a task chain, you don't have to make every continuation task-based or enclose every operation that might throw within a `try...catch` block. Instead, you can add a task-based continuation at the end of the chain and handle all errors there. Any exception—this includes a [task_canceled](#) exception—will propagate down the task chain and bypass any value-based continuations, so that you can handle it in the error-handling task-based continuation. We can rewrite the previous example to use an error-handling task-based continuation:

C++

```
#include <ppltasks.h>
void App::DeleteWithTasksHandleErrors(String^ fileName)
{
    using namespace Windows::Storage;
    using namespace concurrency;

    StorageFolder^ documentsFolder = KnownFolders::DocumentsLibrary;
    auto getFileTask = create_task(documentsFolder->GetFileAsync(fileName));

    getFileTask.then([](StorageFile^ storageFileSample)
    {
        return storageFileSample->DeleteAsync();
    })

    .then([](task<void> t)
    {
        try
        {
```

```

        t.get();
        // .get() didn't throw, so we succeeded.
        OutputDebugString(L"File deleted.");
    }
    catch (Platform::COMException^ e)
    {
        //Example output: The system cannot find the specified file.
        OutputDebugString(e->Message->Data());
    }

    });
}

```

In a task-based continuation, we call the member function `task::get` to get the results of the task. We still have to call `task::get` even if the operation was an `IAsyncAction` that produces no result because `task::get` also gets any exceptions that have been transported down to the task. If the input task is storing an exception, it is thrown at the call to `task::get`. If you don't call `task::get`, or don't use a task-based continuation at the end of the chain, or don't catch the exception type that was thrown, then an `unobserved_task_exception` is thrown when all references to the task have been deleted.

Only catch the exceptions that you can handle. If your app encounters an error that you can't recover from, it's better to let the app crash than to let it continue to run in an unknown state. Also, in general, don't attempt to catch the `unobserved_task_exception` itself. This exception is mainly intended for diagnostic purposes. When `unobserved_task_exception` is thrown, it usually indicates a bug in the code. Often the cause is either an exception that should be handled, or an unrecoverable exception that's caused by some other error in the code.

Managing the thread context

The UI of a UWP app runs in a single-threaded apartment (STA). A task whose lambda returns either an `IAsyncAction` or `IAsyncOperation` is apartment-aware. If the task is created in the STA, then all of its continuations will run also run in it by default, unless you specify otherwise. In other words, the entire task chain inherits apartment-awareness from the parent task. This behavior helps simplify interactions with UI controls, which can only be accessed from the STA.

For example, in a UWP app, in the member function of any class that represents a XAML page, you can populate a `ListBox` control from within a `task::then` method without having to use the `Dispatcher` object.

```

#include <ppltasks.h>
void App::SetFeedText()
{
    using namespace Windows::Web::Syndication;
    using namespace concurrency;
    String^ url =
        "http://windowsteamblog.com/windows_phone/b/wmdev/atom.aspx";
    SyndicationClient^ client = ref new SyndicationClient();
    auto feedOp = client->RetrieveFeedAsync(ref new Uri(url));

    create_task(feedOp).then([this] (SyndicationFeed^ feed)
    {
        m_TextBlock1->Text = feed->Title->Text;
    }));
}

```

If a task doesn't return an [IAsyncAction](#) or [IAsyncOperation](#), then it's not apartment-aware and, by default, its continuations are run on the first available background thread.

You can override the default thread context for either kind of task by using the overload of [task::then](#) that takes a [task_continuation_context](#). For example, in some cases, it might be desirable to schedule the continuation of an apartment-aware task on a background thread. In such a case, you can pass

[task_continuation_context::use_arbitrary](#) to schedule the task's work on the next available thread in a multi-threaded apartment. This can improve the performance of the continuation because its work doesn't have to be synchronized with other work that's happening on the UI thread.

The following example demonstrates when it's useful to specify the [task_continuation_context::use_arbitrary](#) option, and it also shows how the default continuation context is useful for synchronizing concurrent operations on non-thread-safe collections. In this code, we loop through a list of URLs for RSS feeds, and for each URL, we start up an async operation to retrieve the feed data. We can't control the order in which the feeds are retrieved, and we don't really care. When each

[RetrieveFeedAsync](#) operation completes, the first continuation accepts the [SyndicationFeed^](#) object and uses it to initialize an app-defined `FeedData^` object.

Because each of these operations is independent from the others, we can potentially speed things up by specifying the [task_continuation_context::use_arbitrary](#) continuation context. However, after each `FeedData` object is initialized, we have to add it to a [Vector](#), which is not a thread-safe collection. Therefore, we create a continuation and specify [\[task_continuation_context::use_current\]](#)

(/cpp/parallel/concrt/reference/task-continuation-context-class?view=vs-2017& -view=true) to ensure that all the calls to [Append](#) occur in the same Application Single-Threaded Apartment (ASTA) context. Because [task_continuation_context::use_default](#) is

the default context, we don't have to specify it explicitly, but we do so here for the sake of clarity.

C++

```
#include <ppltasks.h>
void App::InitDataSource(Vector<Object^>^ feedList, vector<wstring> urls)
{
    using namespace concurrency;
    SyndicationClient^ client = ref new SyndicationClient();

    std::for_each(std::begin(urls), std::end(urls), [=,this] (std::wstring
url)
    {
        // Create the async operation. feedOp is an
        // IAsyncOperationWithProgress<SyndicationFeed^, RetrievalProgress>^
        // but we don't handle progress in this example.

        auto feedUri = ref new Uri(ref new String(url.c_str()));
        auto feedOp = client->RetrieveFeedAsync(feedUri);

        // Create the task object and pass it the async operation.
        // SyndicationFeed^ is the type of the return value
        // that the feedOp operation will eventually produce.

        // Then, initialize a FeedData object by using the feed info. Each
        // operation is independent and does not have to happen on the
        // UI thread. Therefore, we specify use_arbitrary.
        create_task(feedOp).then([this] (SyndicationFeed^ feed) ->
FeedData^
        {
            return GetFeedData(feed);
        }, task_continuation_context::use_arbitrary())

        // Append the initialized FeedData object to the list
        // that is the data source for the items collection.
        // This all has to happen on the same thread.
        // By using the use_default context, we can append
        // safely to the Vector without taking an explicit lock.
        .then([feedList] (FeedData^ fd)
        {
            feedList->Append(fd);
            OutputDebugString(fd->Title->Data());
        }, task_continuation_context::use_default())

        // The last continuation serves as an error handler. The
        // call to get() will surface any exceptions that were raised
        // at any point in the task chain.
        .then( [this] (task<void> t)
        {
            try
            {
                t.get();
            }
        })
    })
}
```

```

        catch(Platform::InvalidArgumentException^ e)
        {
            //TODO handle error.
            OutputDebugString(e->Message->Data());
        }
    }); //end task chain

}); //end std::for_each
}

```

Nested tasks, which are new tasks that are created inside a continuation, don't inherit apartment-awareness of the initial task.

Handing progress updates

Methods that support [IAsyncOperationWithProgress](#) or [IAsyncActionWithProgress](#) provide progress updates periodically while the operation is in progress, before it completes. Progress reporting is independent from the notion of tasks and continuations. You just supply the delegate for the object's [Progress](#) property. A typical use of the delegate is to update a progress bar in the UI.

Related topics

- [Creating Asynchronous Operations in C++/CX for UWP apps](#)
- [Visual C++ Language Reference](#)
- [Asynchronous Programming](#)
- [Task Parallelism \(Concurrency Runtime\)](#)
- [concurrency::task](#)

Best practices for using the thread pool

Article • 11/15/2024

This topic describes best practices for working with the thread pool.

Do's

- Use the thread pool to do parallel work in your app.
- Use work items to accomplish extended tasks without blocking the UI thread.
- Create work items that are short-lived and independent. Work items run asynchronously and they can be submitted to the pool in any order from the queue.
- Dispatch updates to the UI thread with the [Windows.UI.Core.CoreDispatcher](#).
- Use [ThreadPoolTimer.CreateTimer](#) instead of the `Sleep` function.
- Use the thread pool instead of creating your own thread management system. The thread pool runs at the OS level with advanced capability and is optimized to dynamically scale according to device resources and activity within the process and across the system.
- In C++, ensure that work item delegates use the agile threading model (C++ delegates are agile by default).
- Use pre-allocated work items when you can't tolerate a resource allocation failure at time of use.

Don'ts

- Don't create periodic timers with a *period* value of <1 millisecond (including 0). This will cause the work item to behave as a single-shot timer.
- Don't submit periodic work items that take longer to complete than the amount of time you specified in the *period* parameter.
- Don't try to send UI updates (other than toasts and notifications) from a work item dispatched from a background task. Instead, use background task progress and completion handlers - for example, [IBackgroundTaskInstance.Progress](#).

- When you use work-item handlers that use the **async** keyword, don't assume that all code in the handler has executed when the complete state has been set on the work item. The thread pool work item may be set to the complete state before all of the code in the handler has executed. Code following an **await** keyword within the handler may execute after the work item has been set to the complete state.
- Don't try to run a pre-allocated work item more than once without reinitializing it.
[Create a periodic work item](#)

Related topics

- [Create a periodic work item](#)
- [Submit a work item to the thread pool](#)
- [Use a timer to submit a work item](#)

Feedback

Was this page helpful?

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Call asynchronous APIs in C# or Visual Basic

Article • 10/20/2022

The Universal Windows Platform (UWP) includes many asynchronous APIs to ensure that your app remains responsive when it does work that might take an extended amount of time. This topic discusses how to use asynchronous methods from the UWP in C# or Microsoft Visual Basic.

Asynchronous APIs keep your app from waiting for large operations to complete before continuing execution. For example, an app that downloads info from the Internet might spend several seconds waiting for the info to arrive. If you use a synchronous method to retrieve the info, the app is blocked until the method returns. The app won't respond to user interaction and because it seems non-responsive, the user might become frustrated. By providing asynchronous APIs, the UWP helps to ensure that your app stays responsive to the user when it's performing long operations.

Most of the asynchronous APIs in the UWP don't have synchronous counterparts, so you need to be sure to understand how to use the asynchronous APIs with C# or Visual Basic in your Universal Windows Platform (UWP) app. Here we show how to call asynchronous APIs of the UWP.

Using asynchronous APIs

By convention, asynchronous methods are given names that end in "Async". You typically call asynchronous APIs in response to a user's action, such as when the user clicks a button. Calling an asynchronous method in an event handler is one of the simplest ways of using asynchronous APIs. Here we use the **await** operator as an example.

Suppose that you have an app that lists the titles of blog posts from a certain location. The app has a **Button** that the user clicks to get the titles. The titles are displayed in a **TextBlock**. When the user clicks the button, it is important that the app remains responsive while it waits for the info from the blog's website. To ensure this responsiveness, the UWP provides an asynchronous method, **SyndicationClient.RetrieveFeedAsync**, to download the feed.

The example here gets the lists of blog posts from a blog by calling the asynchronous method, **SyndicationClient.RetrieveFeedAsync**, and awaiting the result.

C#

```
// Put the keyword async on the declaration of the event handler.
private async void Button_Click_1(object sender, RoutedEventArgs e)
{
```

```

Windows.Web.Syndication.SyndicationClient client = new SyndicationClient();

Uri feedUri
    = new
Uri("http://windowsteamblog.com/windows/b/windowsexperience/atom.aspx");

try
{
    SyndicationFeed feed = await client.RetrieveFeedAsync(feedUri);

    // The rest of this method executes after await RetrieveFeedAsync
    completes.
    rssOutput.Text = feed.Title.Text + Environment.NewLine;

    foreach (SyndicationItem item in feed.Items)
    {
        rssOutput.Text += item.Title.Text + ", " +
            item.PublishedDate.ToString() +
Environment.NewLine;
    }
}
catch (Exception ex)
{
    // Log Error.
    rssOutput.Text =
        "I'm sorry, but I couldn't load the page," +
        " possibly due to network problems." +
        "Here's the error message I received: "
        + ex.ToString();
}
}

```

VB

```

' Put the keyword Async on the declaration of the event handler.
Private Async Sub Button_Click_1(sender As Object, e As RoutedEventArgs)
    Dim client As New Windows.Web.Syndication.SyndicationClient()
    Dim feedUri As New
Uri("http://windowsteamblog.com/windows/b/windowsexperience/atom.aspx")

    Try
        Dim feed As SyndicationFeed = Await client.RetrieveFeedAsync(feedUri)

        ' The rest of this method executes after the await operation completes.
        rssOutput.Text = feed.Title.Text & vbCrLf

        For Each item In feed.Items
            rssOutput.Text += $"{item.Title.Text},
{item.PublishedDate.ToString()}{vbCrLf}"
        Next

    Catch ex As Exception
        ' Log Error.
        rssOutput.Text = "I'm sorry, but I couldn't load the page," &

```

```
        " possibly due to network problems." &  
        "Here's the error message I received: " &  
        ex.ToString()  
  
    End Try  
  
End Sub
```

There are a couple of important things about this example. First, the line, `SyndicationFeed feed = await client.RetrieveFeedAsync(feedUri)` uses the **await** operator with the call to the asynchronous method, `RetrieveFeedAsync`. You can think of the **await** operator as telling the compiler that you are calling an asynchronous method, which causes the compiler to do some extra work so you don't have to. Next, the declaration of the event handler includes the keyword **async**. You must include this keyword in the method declaration of any method in which you use the **await** operator.

In this topic, we won't go into a lot of the details of what the compiler does with the **await** operator, but let's examine what your app does so that it is asynchronous and responsive. Consider what happens when you use synchronous code. For example, suppose that there is a method called `SyndicationClient.RetrieveFeed` that is synchronous. (There is no such method, but imagine that there is.) If your app included the line `SyndicationFeed feed = client.RetrieveFeed(feedUri)`, instead of `SyndicationFeed feed = await client.RetrieveFeedAsync(feedUri)`, execution of the app would stop until the return value of `RetrieveFeed` is available. And while your app waits for the method to complete, it can't respond to any other events, such another `Click` event. That is, your app would be blocked until `RetrieveFeed` returns.

But if you call `client.RetrieveFeedAsync`, the method initiates the retrieval and immediately returns. When you use **await** with `RetrieveFeedAsync`, the app temporarily exits the event handler. Then it can process other events while `RetrieveFeedAsync` executes asynchronously. This keeps the app responsive to the user. When `RetrieveFeedAsync` completes and the `SyndicationFeed` is available, the app essentially reenters the event handler where it left off, after `SyndicationFeed feed = await client.RetrieveFeedAsync(feedUri)`, and finishes the rest of the method.

The nice thing about using the **await** operator is that the code doesn't look much different from how the code looks if you used the imaginary `RetrieveFeed` method. There are ways to write asynchronous code in C# or Visual Basic without the **await** operator, but the resulting code tends to emphasize the mechanics of executing asynchronously. This makes asynchronous code hard to write, hard to understand, and hard to maintain. By using the **await** operator, you get the benefits of an asynchronous app without making your code complex.

Return types and results of asynchronous APIs

If you followed the link to [RetrieveFeedAsync](#), you might have noticed that the return type of `RetrieveFeedAsync` is not a [SyndicationFeed](#). Instead, the return type is `IAsyncOperationWithProgress<SyndicationFeed, RetrievalProgress>`. Viewed from the raw syntax, an asynchronous API returns an object that contains the result within it. While it is common, and sometimes useful, to think of an asynchronous method as being awaitable, the `await` operator actually operates on the method's return value, not on the method. When you apply the `await` operator, what you get back is the result of calling `GetResult` on the object returned by the method. In the example, the `SyndicationFeed` is the result of `RetrieveFeedAsync.GetResult()`.

When you use an asynchronous method, you can examine the signature to see what you'll get back after awaiting the value returned from the method. All asynchronous APIs in the UWP return one of the following types:

- [IAsyncOperation<TResult>](#)
- [IAsyncOperationWithProgress<TResult, TProgress>](#)
- [IAsyncAction](#)
- [IAsyncActionWithProgress<TProgress>](#)

The result type of an asynchronous method is the same as the `TResult` type parameter. Types without a `TResult` don't have a result. You can think of the result as being `void`. In Visual Basic, a [Sub](#) procedure is equivalent to a method with a `void` return type.

The table here gives examples of asynchronous methods and lists the return type and result type of each.

Asynchronous method	Return type	Result type
SyndicationClient.RetrieveFeedAsync	IAsyncOperationWithProgress<SyndicationFeed, RetrievalProgress>	SyndicationFeed
FileOpenPicker.PickSingleFileAsync	IAsyncOperation<StorageFile>	StorageFile
XmlDocument.SaveToFileAsync	IAsyncAction	<code>void</code>
InkStrokeContainer.LoadAsync	IAsyncActionWithProgress<UInt64>	<code>void</code>
DataReader.LoadAsync	DataReaderLoadOperation , a custom results class that implements IAsyncOperation<UInt32>	UInt32

Asynchronous methods that are defined in [.NET for UWP apps](#) [↗](#) have the return type `Task` or `Task<TResult>`. Methods that return `Task` are similar to the asynchronous methods in the UWP that return [IAsyncAction](#). In each case, the result of the asynchronous method is `void`. The return type `Task<TResult>` is similar to [IAsyncOperation<TResult>](#) in that the result of the asynchronous method when running the task is the same type as the `TResult` type

parameter. For more info about using **.NET for UWP apps** and tasks, see [.NET for Windows Runtime apps overview](#).

Handling errors

When you use the **await** operator to retrieve your results from an asynchronous method, you can use a **try/catch** block to handle errors that occur in asynchronous methods, just as you do for synchronous methods. The previous example wraps the **RetrieveFeedAsync** method and **await** operation in a **try/catch** block to handle errors when an exception is thrown.

When asynchronous methods call other asynchronous methods, any asynchronous method that results in an exception will be propagated to the outer methods. This means that you can put a **try/catch** block on the outer-most method to catch errors for the nested asynchronous methods. Again, this is similar to how you catch exceptions for synchronous methods. However, you can't use **await** in the **catch** block.

Tip Starting with C# in Microsoft Visual Studio 2005, you can use **await** in the **catch** block.

Summary and next steps

The pattern of calling an asynchronous method that we show here is the simplest one to use when you call asynchronous APIs in an event handler. You can also use this pattern when you call an asynchronous method in an overridden method that returns **void** or a **Sub** in Visual Basic.

As you encounter asynchronous methods in the UWP, it is important to remember:

- By convention, asynchronous methods are given names that end in "Async".
- Any method that uses the **await** operator must have its declaration marked with the **async** keyword.
- When an app finds the **await** operator, the app remains responsive to user interaction while the asynchronous method executes.
- Awaiting the value returned by an asynchronous method returns an object that contains the result. In most cases, the result contained within the return value is what's useful, not the return value itself. You can find the type of the value that is contained inside the result by looking at the return type of the async method.
- Using asynchronous APIs and **async** patterns is often a way to improve the responsiveness of your app.

The example in this topic outputs text that looks like this.

Windows Experience Blog

PC Snapshot: Sony VAIO Y, 8/9/2011 10:26:56 AM -07:00

Tech Tuesday Live Twitter #Chat: Too Much Tech #win7tech, 8/8/2011 12:48:26 PM -07:00

Windows 7 themes: what's new and what's popular!, 8/4/2011 11:56:28 AM -07:00

PC Snapshot: Toshiba Satellite A665 3D, 8/2/2011 8:59:15 AM -07:00

Time for new school supplies? Find back-to-school deals on Windows 7 PCs and Office 2010, 8/1/2011 2:14:40 PM -07:00

Best PCs for blogging (or working) on the go, 8/1/2011 10:08:14 AM -07:00

Tech Tuesday - Blogging Tips and Tricks-#win7tech, 8/1/2011 9:35:54 AM -07:00

PC Snapshot: Lenovo IdeaPad U460, 7/29/2011 9:23:05 AM -07:00

GIVEAWAY: Survive BlogHer with a Sony VAIO SA and a Samsung Focus, 7/28/2011 7:27:14 AM -07:00

3 Ways to Stay Cool This Summer, 7/26/2011 4:58:23 PM -07:00

Getting RAW support in Photo Gallery & Windows 7 (...and a contest!), 7/26/2011 10:40:51 AM -07:00

Tech Tuesdays Live Twitter Chats: Photography Tips, Tricks and Essentials, 7/25/2011 12:33:06 PM -07:00

3 Tips to Go Green With Your PC, 7/22/2011 9:19:43 AM -07:00

How to: Buy a Green PC, 7/22/2011 9:13:22 AM -07:00

Windows 7 themes: the distinctive artwork of Cheng Ling, 7/20/2011 9:53:07 AM -07:00

Create a periodic work item

Article • 10/20/2022

Important APIs

- [CreatePeriodicTimer](#)
- [ThreadPoolTimer](#)

Learn how to create a work item that repeats periodically.

Create the periodic work item

Use the [CreatePeriodicTimer](#) method to create a periodic work item. Supply a lambda that accomplishes the work, and use the *period* parameter to specify the interval between submissions. The period is specified using a [TimeSpan](#) structure. The work item will be resubmitted every time the period elapses, so make sure the period is long enough for work to complete.

[CreateTimer](#) returns a [ThreadPoolTimer](#) object. Store this object in case the timer needs to be canceled.

Note Avoid specifying a value of zero (or any value less than one millisecond) for the interval. This causes the periodic timer to behave as a single-shot timer instead.

Note You can use [CoreDispatcher.RunAsync](#) to access the UI and show progress from the work item.

The following example creates a work item that runs once every 60 seconds:

C#

```
TimeSpan period = TimeSpan.FromSeconds(60);

ThreadPoolTimer PeriodicTimer = ThreadPoolTimer.CreatePeriodicTimer((source)
=>
{
    //
    // TODO: Work
    //

    //
    // Update the UI thread by using the UI core dispatcher.
    //
    Dispatcher.RunAsync(CoreDispatcherPriority.High,
```

```

        () =>
        {
            //
            // UI components can be accessed within this scope.
            //

        });

    }, period);

```

C++

```

    TimeSpan period;
    period.Duration = 60 * 10000000; // 10,000,000 ticks per second

    ThreadPoolTimer ^ PeriodicTimer = ThreadPoolTimer::CreatePeriodicTimer(
        ref new TimerElapsedHandler([this](ThreadPoolTimer^ source)
        {
            //
            // TODO: Work
            //

            //
            // Update the UI thread by using the UI core dispatcher.
            //
            Dispatcher->RunAsync(CoreDispatcherPriority::High,
                ref new DispatchedHandler([this]()
                {
                    //
                    // UI components can be accessed within this scope.
                    //

                }));

        })), period);

```

Handle cancellation of the periodic work item (optional)

If needed, you can handle cancellation of the periodic timer with a [TimerDestroyedHandler](#). Use the [CreatePeriodicTimer](#) overload to supply an additional lambda that handles cancellation of the periodic work item.

The following example creates a periodic work item that repeats every 60 seconds and it also supplies a cancellation handler:

C#

```

using Windows.System.Threading;

    TimeSpan period = TimeSpan.FromSeconds(60);

    ThreadPoolTimer PeriodicTimer =
ThreadPoolTimer.CreatePeriodicTimer((source) =>
    {
        //
        // TODO: Work
        //

        //
        // Update the UI thread by using the UI core dispatcher.
        //
        Dispatcher.RunAsync(CoreDispatcherPriority.High,
            () =>
            {
                //
                // UI components can be accessed within this scope.
                //

            });
    },
    period,
    (source) =>
    {
        //
        // TODO: Handle periodic timer cancellation.
        //

        //
        // Update the UI thread by using the UI core dispatcher.
        //
        Dispatcher->RunAsync(CoreDispatcherPriority.High,
            ()=>
            {
                //
                // UI components can be accessed within this scope.
                //

                // Periodic timer cancelled.

            }));
    });
});

```

C++

```

using namespace Windows::System::Threading;
using namespace Windows::UI::Core;

TimeSpan period;
period.Duration = 60 * 10000000; // 10,000,000 ticks per second

```